

trả lời chi tiết nguyên lý làm sao để đảm bảo tính đúng đắn của dữ liệu khi hệ thống microservice có lượng traffic tăng cao? làm sao đảm bảo dữ liệu không bị xử lý trùng lặp. hãy trả lời chi tiết từ ý tưởng, khái niệm, thiết kế, best practice trong các dự án lớn thực tế. lưu ý, đối với các thuật ngữ hoặc keyword đáng chú ý thì note lại ở phía dưới thành 1 danh sách có giải thích các khái niệm đó

Mình sẽ trả lời theo góc nhìn thiết kế hệ thống thực tế: bắt đầu từ vấn đề “đúng dữ liệu” là gì, sau đó đi vào pattern xử lý trùng lặp, transaction, message queue, retry, idempotency, và cuối cùng là checklist best practice.

Đã suy nghĩ trong giây lát >

Trong microservice traffic cao, vấn đề không chỉ là “server có chịu tải nổi không”, mà là **dữ liệu có còn đúng không khi hàng nghìn request chạy song song, retry liên tục, message bị gửi lại, service bị timeout, database bị lock, queue bị lag, hoặc một service chết giữa chừng.**

Ở hệ thống lớn, ta gần như không bao giờ giả định rằng: “mỗi request chỉ chạy đúng một lần”. Thực tế phải thiết kế theo tư duy:

Một request có thể bị gửi nhiều lần, một message có thể bị consume nhiều lần, một transaction có thể fail giữa chừng, và network có thể timeout dù bên kia đã xử lý thành công.

Vì vậy, để đảm bảo dữ liệu đúng, hệ thống phải được thiết kế để **xử lý lặp lại nhưng không gây sai dữ liệu.**

1. Trước hết phải hiểu “đúng dữ liệu” là đúng cái gì?

Trong hệ thống microservice, “data correctness” thường gồm mấy ý rất quan trọng.

Ví dụ hệ thống banking hoặc payment:

Khách hàng chuyển 1 triệu từ tài khoản A sang tài khoản B.

Dữ liệu đúng nghĩa là:

A bị trừ đúng 1 triệu, không bị trừ 2 lần.

B được cộng đúng 1 triệu, không bị cộng thiếu hoặc cộng dư.

Giao dịch phải có trạng thái rõ ràng: pending, success, failed, reversed.

Nếu hệ thống retry vì timeout, không được tạo thêm giao dịch mới.

Nếu service gửi message nhiều lần, bên nhận không được xử lý trùng.

Nếu có lỗi giữa chừng, hệ thống phải biết rollback, compensate, hoặc đưa giao dịch vào trạng thái cần xử lý tiếp.

Trong monolith, việc này dễ hơn vì nhiều thao tác nằm trong cùng một database transaction. Nhưng trong microservice, mỗi service thường có database riêng. Lúc đó không thể đơn giản dùng một transaction bao trùm toàn bộ hệ thống.

Ví dụ:

Payment Service lưu giao dịch.

Wallet Service trừ tiền.

Notification Service gửi thông báo.

Ledger Service ghi sổ kế toán.

Mỗi service có database riêng. Nếu Payment xử lý xong nhưng Ledger chết thì sao? Nếu Wallet trừ tiền xong nhưng Payment timeout thì sao? Nếu message gửi qua Kafka bị consume 2 lần thì sao?

Đây là lý do microservice cần thiết kế cẩn thận quanh các khái niệm như **idempotency**, **transaction boundary**, **eventual consistency**, **outbox pattern**, **saga**, **optimistic locking**, **distributed lock**, **message deduplication**.

2. Nguyên tắc quan trọng nhất: không tin rằng request/message chỉ đến một lần

Trong hệ thống traffic cao, việc xử lý trùng lặp xảy ra rất thường xuyên.

Một số nguyên nhân thực tế:

Client bấm nút thanh toán nhiều lần.

Mobile app bị mất mạng rồi retry.

API Gateway timeout nhưng service phía sau vẫn xử lý thành công.

Load balancer gửi lại request.

Message queue như Kafka, RabbitMQ, SQS có thể deliver lại message.

Consumer xử lý xong business logic nhưng chưa kịp commit offset thì bị crash.

Service gọi service khác bị timeout, service gọi lại lần nữa.

Cron job hoặc batch job chạy lại cùng một dữ liệu.

Vì vậy, hệ thống lớn không thiết kế theo kiểu:

"Đừng để request bị gửi trùng."

Mà thiết kế theo kiểu:

"Dù request bị gửi trùng, hệ thống vẫn chỉ tạo ra một kết quả đúng."

Đó chính là tư duy **idempotent design**.

3. Idempotency: chìa khóa để chống xử lý trùng lặp

Idempotency nghĩa là cùng một request được gọi nhiều lần thì kết quả cuối cùng vẫn giống như gọi một lần.

Ví dụ không idempotent:

```
POST /transfer
{
  "from": "A",
  "to": "B",
  "amount": 1000000
}
```



Nếu client gửi request này 2 lần, hệ thống có thể tạo 2 giao dịch và trừ tiền 2 lần.

Thiết kế tốt hơn:

```
POST /transfer
Headers:
  Idempotency-Key: abc-123

Body:
{
```



```

    "from": "A",
    "to": "B",
    "amount": 1000000
  }

```

Khi request đầu tiên đến, hệ thống lưu Idempotency-Key = abc-123 .

Nếu cùng key này được gửi lại, hệ thống không xử lý lại business logic nữa. Nó trả về kết quả cũ.

Luồng xử lý thực tế:

Client gửi request transfer với Idempotency-Key = abc-123

Payment Service kiểm tra:
- Key này đã tồn tại chưa?

Nếu chưa:

- Tạo record idempotency
- Xử lý giao dịch
- Lưu kết quả
- Trả response

Nếu rồi:

- Không xử lý lại
- Trả lại response cũ

Bảng database có thể như sau:

idempotency_key	request_hash	status	response_body	created_at
abc-123	xxx	SUCCESS	{...}	2026-06-23

Điểm cực kỳ quan trọng: **không chỉ lưu key, mà nên lưu cả request hash.**

Vì có trường hợp client dùng lại cùng một key nhưng body khác:

Lần 1:

Idempotency-Key: abc-123
amount = 1000000

Lần 2:

Idempotency-Key: abc-123
amount = 2000000

Nếu chỉ dựa vào key thì rất nguy hiểm. Do đó hệ thống nên lưu `request_hash`. Nếu cùng key nhưng body khác, trả lỗi kiểu:

409 Conflict – Same idempotency key but different request body



Trong dự án thực tế, idempotency thường áp dụng cho các API kiểu:

Tạo payment.

Tạo order.

Chuyển tiền.

Refund.

Đặt vé.

Tạo transaction.

Submit form quan trọng.

Không nhất thiết mọi API đều cần idempotency. Ví dụ `GET /users/1` vốn đã an toàn.

Nhưng các API có side effect như `POST /payment`, `POST /order`, `POST /transfer` thì gần như bắt buộc phải nghĩ đến idempotency.

4. Unique constraint ở database: lớp bảo vệ cuối cùng cực kỳ quan trọng

Rất nhiều người chỉ xử lý chống trùng ở code:

```
if (!exists(transactionId)) {  
    insert(transaction);  
}
```

Nhìn thì đúng, nhưng trong traffic cao, đoạn này có race condition.

Ví dụ hai request giống nhau đến cùng lúc:

Thread 1: kiểm tra chưa có transactionId
Thread 2: kiểm tra chưa có transactionId

Thread 1: insert
Thread 2: insert



Kết quả là insert trùng.

Cách đúng là database phải có **unique constraint**.

Ví dụ:

```
CREATE UNIQUE INDEX uk_payment_request_id  
ON payments(request_id);
```

Hoặc:

```
CREATE UNIQUE INDEX uk_idempotency_key  
ON idempotency_records(idempotency_key);
```

Lúc đó dù 2 thread chạy song song, database chỉ cho một request insert thành công. Request còn lại sẽ bị lỗi duplicate key. Code bắt lỗi duplicate key rồi đọc lại record cũ để trả kết quả.

Thiết kế tốt thường là:

Application kiểm tra logic
+
Database enforce uniqueness



Không nên chỉ tin application.

Trong hệ thống lớn, database constraint là lớp phòng thủ cuối cùng để bảo vệ dữ liệu khỏi race condition.

5. Transaction boundary: mỗi service phải có ranh giới transaction rõ ràng

Trong microservice, một service không nên sửa trực tiếp database của service khác.

Ví dụ không nên:

Payment Service ghi vào DB của Payment
 Payment Service ghi luôn vào DB của Wallet
 Payment Service ghi luôn vào DB của Ledger



Vì như vậy phá vỡ nguyên tắc ownership dữ liệu.

Thiết kế đúng hơn:

Payment Service chỉ quản lý payment DB
 Wallet Service chỉ quản lý wallet DB
 Ledger Service chỉ quản lý ledger DB



Mỗi service có một **transaction boundary** riêng.

Ví dụ trong Wallet Service , việc trừ tiền phải nằm trong một database transaction local:

BEGIN TRANSACTION



1. Kiểm tra transaction_id đã xử lý chưa
2. Kiểm tra số dư
3. Trừ tiền
4. Ghi wallet_transaction
5. Commit

COMMIT

Điểm quan trọng là thao tác:

trừ tiền + ghi transaction đã xử lý



phải nằm trong cùng một transaction.

Nếu trừ tiền xong nhưng chưa ghi transaction log mà service chết, hệ thống không biết giao dịch đó đã xử lý chưa. Khi retry có thể trừ lần nữa.

Thiết kế tốt hơn:

BEGIN;

INSERT INTO processed_messages(message_id, status)
VALUES ('msg-001', 'PROCESSING');

UPDATE wallets
SET balance = balance - 1000000

```
WHERE account_id = 'A'
AND balance >= 1000000;

INSERT INTO wallet_transactions(transaction_id, account_id, amount, type)
VALUES ('tx-001', 'A', 1000000, 'DEBIT');

UPDATE processed_messages
SET status = 'SUCCESS'
WHERE message_id = 'msg-001';

COMMIT;
```

Nghĩa là phải lưu dấu vết xử lý trong cùng transaction với thay đổi dữ liệu chính.

6. Message queue không đảm bảo “exactly once” theo cách nhiều người tưởng

Trong thực tế, với Kafka, RabbitMQ, SQS, Pub/Sub, ta thường phải thiết kế theo giả định:

Message có thể được deliver nhiều hơn một lần.

Có khái niệm “exactly once” trong một số nền tảng, ví dụ Kafka có Exactly-Once Semantics trong một số điều kiện. Nhưng ở góc độ business, đặc biệt khi message dẫn đến update database bên ngoài Kafka, ta vẫn nên thiết kế consumer theo kiểu **idempotent consumer**.

Ví dụ message:

```
{
  "messageId": "msg-001",
  "transactionId": "tx-001",
  "accountId": "A",
  "amount": 1000000,
  "type": "DEBIT"
}
```

Consumer nhận message này có thể xử lý như sau:

1. Check messageId hoặc transactionId đã xử lý chưa
2. Nếu đã xử lý rồi thì bỏ qua
3. Nếu chưa xử lý thì xử lý business
4. Ghi lại rằng message này đã xử lý
5. Commit transaction
6. Commit offset / ack message



Thứ tự rất quan trọng.

Không nên ack message trước rồi mới xử lý database.

Vì nếu ack trước, sau đó service chết, message mất nhưng dữ liệu chưa được update.

Cũng không nên xử lý database xong mà không có deduplication record.

Vì nếu xử lý xong nhưng chưa ack, message sẽ bị deliver lại và có thể xử lý trùng.

Pattern thường dùng:

Process message + save processed_message trong cùng DB transaction
Sau khi commit DB thành công thì mới ack message



Nếu message bị gửi lại, consumer kiểm tra processed_message thấy đã có rồi thì không xử lý nữa.

7. Outbox Pattern: chống mất message khi vừa ghi DB vừa publish event

Một lỗi rất phổ biến trong microservice:

1. Payment Service lưu payment SUCCESS vào database
2. Payment Service publish event PaymentSucceeded



Nếu bước 1 thành công nhưng bước 2 fail thì sao?

Database đã ghi payment success, nhưng các service khác như Notification, Ledger, Shipping không nhận được event.

Ngược lại:

1. Publish event thành công
2. Lưu database fail



Service khác nhận event nhưng dữ liệu chính chưa commit.

Đây là vấn đề kinh điển: **dual-write problem**.

Cách giải quyết thực tế là dùng **Outbox Pattern**.

Thay vì vừa ghi DB vừa publish trực tiếp ra queue, service ghi event vào một bảng outbox trong cùng database transaction với dữ liệu chính.

Ví dụ:

```
BEGIN TRANSACTION
```

1. Insert payment record
2. Insert outbox event PaymentCreated

```
COMMIT
```



Bảng outbox:

event_id	aggregate_id	event_type	payload	status
evt-001	payment-001	PaymentCreated	{...}	NEW



Sau đó có một background worker hoặc Debezium CDC đọc bảng outbox và publish event ra Kafka/RabbitMQ.

Luồng:

```
Payment API
```

```
|
|-- transaction DB
|   |-- insert payment
|   |-- insert outbox_event
```

```
COMMIT
```

```
Outbox Publisher
```

```
|
|-- đọc outbox_event NEW
|-- publish Kafka
|-- mark PUBLISHED
```



Ưu điểm là payment và event được ghi atomically trong cùng một local transaction. Nếu transaction fail thì cả payment và event đều không có. Nếu transaction success thì event chắc chắn nằm trong outbox và có thể được publish lại cho đến khi thành công.

Trong dự án lớn, Outbox Pattern là một trong những pattern quan trọng nhất để đảm bảo dữ liệu nhất quán khi dùng event-driven architecture.

8. Saga Pattern: xử lý giao dịch dài qua nhiều service

Trong microservice, không nên cố gắng dùng distributed transaction kiểu 2PC cho mọi thứ vì rất nặng, khó scale, dễ lock lâu và làm hệ thống phức tạp.

Thay vào đó, các hệ thống lớn thường dùng **Saga Pattern**.

Saga chia một business transaction lớn thành nhiều local transaction nhỏ.

Ví dụ order flow:

1. Order Service tạo order PENDING
2. Payment Service charge tiền
3. Inventory Service giữ hàng
4. Shipping Service tạo shipment
5. Order Service chuyển order thành CONFIRMED



Nếu bước 3 fail thì sao?

Ví dụ Payment đã charge tiền nhưng Inventory giữ hàng thất bại.

Saga không rollback kiểu database transaction truyền thống. Nó dùng **compensating transaction**.

Tức là tạo một hành động bù:

Inventory giữ hàng fail
=> Payment Service refund tiền
=> Order Service chuyển order thành FAILED



Có hai kiểu Saga chính.

8.1. Choreography Saga

Mỗi service lắng nghe event và tự phản ứng.

Ví dụ:

OrderCreated
-> Payment Service charge
PaymentSucceeded
-> Inventory Service reserve
InventoryReserved
-> Shipping Service create shipment
ShippingCreated
-> Order Service confirm order



Ưu điểm:

Hệ thống ít phụ thuộc vào một orchestrator trung tâm.

Service tương đối độc lập.

Phù hợp flow đơn giản.

Nhược điểm:

Khó quan sát toàn bộ luồng.

Khi flow phức tạp, event chồng chéo khó debug.

Dễ thành "event spaghetti".

8.2. Orchestration Saga

Có một service trung tâm điều phối luồng.

Ví dụ Order Orchestrator :

Order Orchestrator:

1. Gọi Payment Service
2. Nếu success, gọi Inventory Service
3. Nếu success, gọi Shipping Service
4. Nếu bước nào fail, gọi các bước compensate tương ứng



Ưu điểm:

Dễ nhìn toàn bộ flow.

Dễ quản lý trạng thái.

Dễ retry từng bước.

Phù hợp hệ thống banking, payment, order phức tạp.

Nhược điểm:

Orchestrator trở thành thành phần quan trọng, phải thiết kế high availability.

Có thể tạo coupling giữa các service.

Trong hệ thống tài chính, orchestration thường dễ kiểm soát hơn vì cần audit, trạng thái rõ ràng, retry có kiểm soát, và khả năng điều tra khi lỗi.

9. Optimistic Locking: chống update đè nhau khi traffic cao

Giả sử tài khoản A có số dư 1 triệu.

Hai request rút tiền cùng lúc, mỗi request rút 800k.

Nếu xử lý không cẩn thận:

Request 1 đọc balance = 1,000,000

Request 2 đọc balance = 1,000,000

Request 1 trừ còn 200,000

Request 2 cũng trừ còn 200,000

Kết quả sai, vì tổng rút là 1.6 triệu nhưng hệ thống chỉ ghi còn 200k.

Một cách xử lý là dùng **optimistic locking** với cột `version`.

Bảng wallet:

account_id	balance	version
A	1000000	1

Khi update:

```
UPDATE wallets
SET balance = balance - 800000,
    version = version + 1
WHERE account_id = 'A'
    AND version = 1
    AND balance >= 800000;
```

Request 1 update thành công, version từ 1 thành 2.

Request 2 vẫn đang nghĩ version là 1, update sẽ fail vì DB không còn record version = 1 nữa.

Khi đó Request 2 phải đọc lại balance mới và quyết định tiếp:

Balance hiện tại = 200,000

Không đủ để rút 800,000

=> reject giao dịch

Optimistic locking phù hợp khi conflict không quá cao. Nếu conflict quá cao, có thể cần thiết kế khác như queue theo account, partition theo key, hoặc dùng pessimistic lock trong một số đoạn quan trọng.

10. Pessimistic Locking: khóa dữ liệu khi cần đảm bảo tuyệt đối

Trong một số nghiệp vụ nhạy cảm, có thể dùng lock ở database:

```
SELECT * FROM wallets  
WHERE account_id = 'A'  
FOR UPDATE;
```

Khi transaction này đang giữ lock, transaction khác muốn update account A phải chờ.

Ưu điểm:

Để đảm bảo không có hai transaction cùng sửa một dòng dữ liệu.

Phù hợp cho nghiệp vụ tiền, số dư, tồn kho quan trọng.

Nhược điểm:

Nếu traffic cao, lock lâu sẽ làm giảm throughput.

Có thể gây deadlock nếu lock nhiều resource sai thứ tự.

Không nên lạm dụng.

Best practice là lock càng ngắn càng tốt. Chỉ lock trong phạm vi transaction nhỏ, không gọi API bên ngoài trong khi đang giữ lock.

Ví dụ sai:

```
BEGIN  
lock wallet  
gọi Payment Gateway bên ngoài  
update wallet  
COMMIT
```



Nếu Payment Gateway chậm 5 giây, wallet bị lock 5 giây. Traffic cao sẽ nghẽn.

Thiết kế tốt hơn:

1. Gọi external service trước hoặc tách flow
2. Khi cần update balance, mở transaction ngắn
3. Lock row
4. Update
5. Commit ngay



11. Distributed Lock: dùng cẩn thận, không phải thuốc tiên

Nhiều người thấy trùng lặp là nghĩ ngay đến Redis lock.

Ví dụ:

```
lock: account:A
```



Rồi xử lý xong thì unlock.

Distributed lock có thể hữu ích trong một số trường hợp, nhưng không nên xem nó là lớp bảo vệ duy nhất cho dữ liệu tài chính.

Vì distributed lock có nhiều vấn đề:

Service giữ lock rồi chết.

Lock hết TTL trong khi process vẫn đang chạy.

Clock drift.

Network partition.

Unlock nhầm lock của request khác nếu không dùng token.

Redis failover có thể gây hành vi khó đoán nếu thiết kế không kỹ.

Nếu dùng Redis lock, phải dùng lock token:

```
SET lock_key unique_token NX PX 30000
```



Khi unlock, chỉ xóa nếu token khớp.

Không được:

```
DEL lock_key
```



một cách mù quáng, vì có thể xóa lock của process khác.

Trong hệ thống lớn, distributed lock thường chỉ là lớp hỗ trợ. Lớp bảo vệ cuối vẫn nên là database constraint, transaction, idempotency record, versioning.

12. Deduplication: chống xử lý trùng message/event

Để chống message bị xử lý nhiều lần, mỗi event/message cần có ID duy nhất.

Ví dụ:

```
{
  "eventId": "evt-001",
  "eventType": "PaymentSucceeded",
  "aggregateId": "payment-001",
  "occurredAt": "2026-06-23T10:00:00Z",
  "payload": {...}
}
```

Consumer lưu event đã xử lý:

```
processed_events
-----
event_id
consumer_name
processed_at
status
```



Khi nhận event:

```
Nếu event_id + consumer_name đã tồn tại:
    bỏ qua
Nếu chưa:
    xử lý
    lưu processed event
```



Tại sao cần consumer_name ?

Vì cùng một event có thể được xử lý bởi nhiều consumer khác nhau:

Notification Service cần gửi email.

Ledger Service cần ghi sổ.

Analytics Service cần thống kê.

Nếu chỉ lưu event_id, service này có thể làm ảnh hưởng service khác. Vì vậy key thường là:

```
event_id + consumer_name
```



Hoặc mỗi service có bảng processed riêng trong DB của nó.

13. Consumer phải idempotent, không chỉ producer

Nhiều hệ thống chỉ tập trung chống gửi trùng ở producer, nhưng quên rằng consumer cũng phải tự bảo vệ.

Producer có thể gửi event một lần, nhưng queue vẫn có thể deliver lại.

Vì vậy consumer phải xử lý như sau:

Event PaymentSucceeded tx-001 đến Ledger Service



Ledger kiểm tra:

– Ledger entry cho tx-001 đã tồn tại chưa?

Nếu chưa:

- Ghi ledger debit/credit
- Mark event processed

Nếu rồi:

- Không ghi lại

Bảng ledger nên có unique constraint:

```
CREATE UNIQUE INDEX uk_ledger_transaction  
ON ledger_entries(transaction_id, entry_type);
```

Ví dụ một giao dịch chuyển tiền có thể có hai ledger entry:

```
tx-001 | DEBIT | account A | -1000000  
tx-001 | CREDIT | account B | +1000000
```



Unique key giúp đảm bảo không ghi debit 2 lần hoặc credit 2 lần cho cùng transaction.

14. Retry phải có kiểm soát, không retry mù quáng

Traffic cao thường đi kèm lỗi timeout, rate limit, connection reset, queue lag. Retry là cần thiết, nhưng retry sai cách có thể làm hệ thống sập nặng hơn.

Ví dụ một downstream service đang quá tải. Nếu tất cả upstream cùng retry liên tục, tải sẽ nhân lên nhiều lần.

Best practice:

Dùng retry với exponential backoff.

Có jitter để tránh nhiều request retry cùng lúc.

Giới hạn số lần retry.

Chỉ retry lỗi tạm thời.

Không retry lỗi business rõ ràng.

Có dead-letter queue cho message fail nhiều lần.

Ví dụ:

```
Retry lần 1 sau 100ms  
Retry lần 2 sau 300ms  
Retry lần 3 sau 1s  
Retry lần 4 sau 3s  
Nếu vẫn fail -> đưa vào DLQ
```



Không nên retry vô hạn.

Đặc biệt với payment, transfer, refund, retry phải dựa trên transaction ID/idempotency key để tránh tạo giao dịch mới.

Ví dụ gọi payment gateway timeout.

Không được tạo request mới với transaction ID mới.

Phải query trạng thái giao dịch cũ hoặc retry với cùng idempotency key.

15. State machine: quản lý trạng thái giao dịch rõ ràng

Với nghiệp vụ quan trọng, không nên chỉ lưu `success = true/false`.

Nên thiết kế trạng thái rõ ràng.

Ví dụ payment:

```
INITIATED  
PROCESSING  
AUTHORIZED  
CAPTURED  
FAILED  
CANCELLED
```



REFUND_PENDING
REFUNDED

Transfer:

CREATED
DEBIT_PENDING
DEBIT_SUCCESS
CREDIT_PENDING
CREDIT_SUCCESS
COMPLETED
FAILED
REVERSAL_PENDING
REVERSED



Mỗi transition phải hợp lệ.

Ví dụ:

CREATED -> PROCESSING -> SUCCESS
CREATED -> PROCESSING -> FAILED
SUCCESS -> REFUND_PENDING -> REFUNDED



Không cho phép trạng thái nhảy lung tung:

FAILED -> SUCCESS
REFUNDED -> CAPTURED



trừ khi có rule rõ ràng.

State machine giúp hệ thống:

Retry đúng bước.

Biết giao dịch đang kẹt ở đâu.

Có thể resume khi service restart.

Dễ audit.

Dễ support production issue.

Ví dụ nếu transfer đang ở DEBIT_SUCCESS nhưng chưa CREDIT_SUCCESS, hệ thống biết cần tiếp tục credit hoặc reverse debit.

16. Eventual Consistency: không phải lúc nào cũng đồng bộ ngay lập tức

Trong microservice, không nên kỳ vọng tất cả service cập nhật cùng lúc ngay lập tức.

Ví dụ Payment Service báo success trước, Notification Service gửi email sau vài giây, Analytics cập nhật dashboard sau vài phút.

Đây là **eventual consistency**.

Tức là dữ liệu giữa các service có thể lệch tạm thời, nhưng cuối cùng sẽ nhất quán.

Quan trọng là phải biết dữ liệu nào bắt buộc consistent ngay, dữ liệu nào có thể eventual.

Ví dụ trong banking:

Số dư khả dụng của tài khoản cần rất chặt.

Ledger cần chính xác tuyệt đối.

Notification có thể trễ.

Analytics có thể trễ.

Email/SMS có thể gửi lại hoặc xử lý sau.

Không nên áp cùng một mức consistency cho mọi thứ. Nếu cái gì cũng đòi strong consistency, hệ thống sẽ rất khó scale.

17. CQRS: tách luồng ghi và luồng đọc khi traffic cao

Ở hệ thống traffic lớn, đọc và ghi có đặc điểm rất khác nhau.

Ví dụ:

Ghi giao dịch cần chính xác, transaction, lock, idempotency.

Đọc lịch sử giao dịch cần nhanh, filter, pagination, cache, search.

Nếu cùng một database phục vụ cả write và read nặng, hệ thống dễ nghẽn.

Có thể dùng **CQRS**:

Command side:

– Xử lý ghi



- Validate business
- Transaction
- Idempotency
- State machine

Query side:

- Tối ưu đọc
- Denormalized view
- Cache
- Search index

Ví dụ khi payment thành công, hệ thống publish event. Query model nhận event và cập nhật bảng đọc:

```
payment_read_model
transaction_history_view
user_activity_view
```



Người dùng xem lịch sử giao dịch từ read model. Nếu read model trễ vài giây, UI có thể hiển thị trạng thái "processing" hoặc đọc trực tiếp từ command side cho giao dịch mới nhất.

CQRS không bắt buộc cho mọi hệ thống, nhưng ở dự án lớn nó rất hữu ích khi traffic đọc cao hơn ghi rất nhiều.

18. Partitioning theo key để giảm conflict

Một kỹ thuật thực tế là xử lý theo cùng một key trên cùng một partition.

Ví dụ Kafka partition theo accountId hoặc orderId .

```
Message của account A -> partition 1
Message của account B -> partition 2
Message của account C -> partition 3
```



Kafka đảm bảo ordering trong cùng một partition.

Nếu tất cả message liên quan đến account A đi cùng partition, consumer sẽ xử lý tuần tự theo account A. Điều này giảm nguy cơ race condition.

Nhưng phải chọn partition key cẩn thận.

Nếu chọn sai key, ví dụ tất cả message dùng key là bankId , toàn bộ traffic dồn vào một partition gây bottleneck.

Nếu nghiệp vụ cần đảm bảo thứ tự theo account, chọn `accountId` .

Nếu cần đảm bảo thứ tự theo order, chọn `orderId` .

Nếu cần đảm bảo thứ tự theo user, chọn `userId` .

Trong hệ thống tài chính, partitioning theo account hoặc transaction aggregate là pattern rất phổ biến.

19. Database isolation level: phải hiểu mình đang dùng mức nào

Database transaction không phải lúc nào cũng đảm bảo như mình tưởng. Isolation level ảnh hưởng rất lớn.

Các mức phổ biến:

`READ COMMITTED` : tránh đọc dữ liệu chưa commit, nhưng vẫn có thể gặp non-repeatable read.

`REPEATABLE READ` : cùng một transaction đọc lại sẽ thấy dữ liệu ổn định hơn, nhưng tùy database vẫn có các hành vi khác nhau.

`SERIALIZABLE` : chặt nhất, gần như transaction chạy tuần tự, nhưng tốn hiệu năng hơn.

Trong thực tế, nhiều hệ thống dùng `READ COMMITTED` kết hợp với:

Unique constraint.

Optimistic locking.

Pessimistic lock khi cần.

Atomic update.

Ví dụ trừ tiền không nên làm kiểu:

1. `SELECT balance`
2. Tính ở application
3. `UPDATE balance`



một cách lỏng lẻo.

Tốt hơn:

```
UPDATE wallets
SET balance = balance - :amount
WHERE account_id = :accountId
AND balance >= :amount;
```

Sau đó kiểm tra số row affected.

Nếu row affected = 1 , trừ tiền thành công.

Nếu row affected = 0 , không đủ tiền hoặc account không tồn tại.

Đây là atomic operation ở database, an toàn hơn nhiều.

20. Audit log và ledger: dữ liệu tài chính không nên chỉ lưu balance

Một lỗi thiết kế rất nguy hiểm là chỉ lưu balance hiện tại:

```
account_id | balance
A          | 5000000
```



Nếu balance sai, không biết sai từ đâu.

Trong hệ thống thực tế, đặc biệt banking/payment, phải có ledger hoặc transaction log.

Thay vì chỉ update balance, cần ghi lại từng entry:

```
ledger_entries
-----
entry_id
transaction_id
account_id
direction
amount
balance_before
balance_after
created_at
```



Ví dụ:

```
tx-001 | A | DEBIT | 1000000 | 5000000 | 4000000
tx-001 | B | CREDIT | 1000000 | 2000000 | 3000000
```



Ledger giúp:

Audit.

Reconcile.

Điều tra lỗi.

Replay dữ liệu.

So sánh tổng debit/credit.

Phát hiện giao dịch lệch.

Trong banking, ledger thường là source of truth. Balance có thể là dữ liệu được tính ra hoặc cache từ ledger, nhưng ledger mới là bằng chứng giao dịch.

21. Reconciliation: hệ thống lớn phải có job đối soát

Dù thiết kế tốt, hệ thống lớn vẫn phải giả định rằng lỗi có thể xảy ra.

Vì vậy cần có reconciliation job.

Ví dụ mỗi đêm hệ thống kiểm tra:

Tổng debit có bằng tổng credit không.

Payment success có ledger entry chưa.

Order paid có invoice chưa.

Wallet transaction có bị thiếu entry không.

Outbox event nào chưa publish quá lâu.

Message nào nằm trong DLQ.

Giao dịch nào stuck ở PROCESSING quá 30 phút.

Ví dụ query kiểm tra:

```
SELECT transaction_id
FROM payments
WHERE status = 'SUCCESS'
AND transaction_id NOT IN (
```



```
SELECT transaction_id FROM ledger_entries  
);
```

Hoặc kiểm tra event outbox chưa publish:

```
SELECT *  
FROM outbox_events  
WHERE status = 'NEW'  
AND created_at < now() - interval '10 minutes';
```

Reconciliation là best practice cực kỳ quan trọng trong banking/payment. Không có hệ thống nào chỉ dựa vào realtime processing rồi tin là vĩnh viễn đúng.

22. Observability: không đo thì không biết hệ thống đang sai

Khi traffic tăng cao, lỗi dữ liệu thường không hiện ra ngay. Nó có thể xuất hiện dưới dạng:

Queue lag tăng.

Retry tăng.

Duplicate key error tăng.

Transaction timeout tăng.

Deadlock tăng.

DLQ tăng.

Payment stuck ở PROCESSING.

Outbox event publish fail.

Consumer xử lý chậm.

Vì vậy phải có observability.

Cần log các ID quan trọng:

```
requestId  
correlationId  
idempotencyKey
```



```
transactionId  
eventId  
messageId  
accountId  
orderId  
traceId
```

Ví dụ một request đi qua nhiều service:

```
API Gateway  
-> Payment Service  
-> Wallet Service  
-> Ledger Service  
-> Notification Service
```



Tất cả service phải log cùng `correlationId`, để khi có lỗi có thể trace toàn bộ luồng.

Metric quan trọng:

Số request duplicate.

Số idempotency hit.

Số duplicate message bị bỏ qua.

Queue lag.

DLQ count.

Retry count.

Transaction failure rate.

Database lock wait time.

Deadlock count.

Outbox pending count.

Saga stuck count.

Không có observability thì khi production sai dữ liệu, team chỉ biết "có lỗi" nhưng không biết lỗi ở service nào, message nào, transaction nào.

23. Một thiết kế thực tế cho hệ thống chuyển tiền

Ví dụ thiết kế transfer trong microservice.

Client gọi:

```
POST /transfers
Idempotency-Key: abc-123
```

Body:

```
{
  "fromAccount": "A",
  "toAccount": "B",
  "amount": 1000000
}
```

Bước 1: API Gateway

Gateway kiểm tra auth, rate limit, validate cơ bản.

Gateway tạo requestId , truyền xuống service.

```
X-Request-Id: req-001
X-Correlation-Id: corr-001
Idempotency-Key: abc-123
```



Bước 2: Transfer Service nhận request

Transfer Service kiểm tra idempotency key.

Nếu key đã xử lý:

Trả response cũ



Nếu key chưa xử lý:

BEGIN TRANSACTION



1. Insert idempotency record abc-123 PROCESSING
2. Insert transfer tx-001 status CREATED
3. Insert outbox event TransferCreated
4. Commit

COMMIT

Response có thể là:

```
{
  "transactionId": "tx-001",
  "status": "CREATED"
}
```

Bước 3: Outbox Publisher publish event

Outbox Publisher đọc event:

```
TransferCreated(tx-001)
```



Publish vào Kafka.

Sau đó mark event là PUBLISHED .

Bước 4: Wallet Service consume event

Wallet Service nhận TransferCreated .

Nó xử lý debit tài khoản A.

```
BEGIN TRANSACTION
```



1. Check eventId đã xử lý chưa
2. Check transactionId đã debit chưa
3. Atomic update balance:
balance = balance - amount
where account_id = A and balance >= amount
4. Insert wallet_transaction DEBIT
5. Insert processed_event
6. Insert outbox event DebitSucceeded hoặc DebitFailed

```
COMMIT
```

Nếu đủ tiền:

```
DebitSucceeded(tx-001)
```



Nếu không đủ tiền:

```
DebitFailed(tx-001)
```



Bước 5: Credit tài khoản B

Khi debit success, Wallet Service hoặc Transfer Orchestrator tiếp tục credit tài khoản B.

BEGIN TRANSACTION



1. Check credit cho tx-001 đã làm chưa
2. Update balance B = balance + amount
3. Insert wallet_transaction CREDIT
4. Mark processed
5. Publish/Outbox CreditSucceeded

COMMIT

Bước 6: Transfer Service hoàn tất

Transfer Service nhận CreditSucceeded .

BEGIN TRANSACTION



1. Check event đã xử lý chưa
2. Update transfer status COMPLETED
3. Update idempotency record SUCCESS + response body
4. Insert processed_event

COMMIT

Nếu client retry lại Idempotency-Key: abc-123 , Transfer Service trả lại:

```
{
  "transactionId": "tx-001",
  "status": "COMPLETED"
}
```

Không tạo transfer mới.

Không trừ tiền lần nữa.

Không cộng tiền lần nữa.

24. Best practice trong dự án lớn

24.1. Luôn có business ID duy nhất

Không dựa vào auto-increment ID nội bộ để chống trùng.

Nên có các ID rõ ràng:

```
requestId
idempotencyKey
transactionId
orderId
paymentId
eventId
messageId
```



Các ID này nên được truyền xuyên suốt hệ thống.

24.2. API có side effect phải idempotent

Các API kiểu tạo giao dịch, tạo đơn, thanh toán, refund, transfer nên hỗ trợ idempotency key.

Không để client retry tạo giao dịch mới.

24.3. Database phải enforce rule quan trọng

Không chỉ validate ở code.

Phải có:

Unique constraint.

Foreign key nếu phù hợp.

Check constraint.

Version column.

Transaction boundary.

Ví dụ:

```
UNIQUE(idempotency_key)
UNIQUE(transaction_id, entry_type)
CHECK(amount > 0)
CHECK(balance >= 0)
```

24.4. Không gọi external API trong lúc đang giữ DB lock

Đây là lỗi production rất hay gặp.

Sai:

```
BEGIN
lock row
call external payment gateway
update DB
COMMIT
```



Đúng hơn:

```
call external service với idempotency key
sau đó mở transaction ngắn để update DB
```



Hoặc dùng saga để tách bước.

24.5. Consumer phải idempotent

Mọi consumer đọc Kafka/RabbitMQ/SQS phải giả định message có thể đến nhiều lần.

Phải có processed table hoặc unique business key.

24.6. Dùng Outbox Pattern cho event quan trọng

Không nên vừa update DB vừa publish queue trực tiếp mà không có cơ chế đảm bảo.

Dữ liệu chính và event nên được ghi trong cùng transaction qua outbox.

24.7. Retry phải đi kèm idempotency

Retry mà không có idempotency là rất nguy hiểm.

Đặc biệt với:

Payment.

Transfer.

Refund.

Booking.

Inventory deduction.

24.8. Có DLQ và cơ chế replay

Message fail nhiều lần không nên làm nghẽn toàn bộ queue.

Đưa vào DLQ, có dashboard, alert, và tool replay sau khi fix lỗi.

24.9. Thiết kế trạng thái rõ ràng

Không dùng boolean đơn giản cho flow phức tạp.

Nên có state machine và transition rule rõ ràng.

24.10. Có reconciliation job

Realtime system không đủ.

Phải có job đối soát định kỳ để tìm giao dịch lệch, event chưa publish, saga stuck, ledger thiếu entry.

25. Sai lầm phổ biến cần tránh

Sai lầm 1: Nghĩ rằng message queue đảm bảo không bao giờ gửi trùng.

Thực tế consumer vẫn phải idempotent.

Sai lầm 2: Dùng `if exists then insert` mà không có unique constraint.

Traffic cao sẽ dính race condition.

Sai lầm 3: Chỉ lưu balance, không lưu ledger.

Khi sai dữ liệu sẽ không audit được.

Sai lầm 4: Retry payment bằng request mới.

Dễ charge tiền khách nhiều lần.

Sai lầm 5: Publish event sau khi commit DB mà không có outbox.

Dễ mất event.

Sai lầm 6: Lock quá rộng, giữ lock khi gọi external API.

Để làm hệ thống nghẽn hoặc deadlock.

Sai lầm 7: Không có correlation ID.

Khi production lỗi không trace được request đi qua service nào.

Sai lầm 8: Không có reconciliation.

Sai dữ liệu nhỏ có thể tích tụ thành lỗi lớn.

26. Kết luận theo góc nhìn senior engineer

Để đảm bảo dữ liệu đúng trong microservice traffic cao, đừng chỉ nghĩ đến scale server, tăng thread, tăng instance. Cốt lõi là phải thiết kế hệ thống theo tư duy:

Request có thể đến nhiều lần.
Message có thể bị deliver lại.
Service có thể chết giữa transaction.
Network timeout không có nghĩa là xử lý thất bại.
Database là lớp bảo vệ cuối cùng.
Business operation phải idempotent.
Mỗi service phải có transaction boundary rõ ràng.
Event quan trọng phải dùng outbox.
Flow nhiều service phải dùng saga/compensation.
Dữ liệu tài chính phải có ledger và audit.
Hệ thống phải có reconciliation để tự phát hiện lệch.



Trong dự án lớn, data correctness không đến từ một kỹ thuật duy nhất. Nó là sự kết hợp của:

Idempotency
Unique constraint
Local transaction
Outbox Pattern
Idempotent Consumer
Saga Pattern
Optimistic/Pessimistic Locking
Retry with backoff
DLQ
State machine
Ledger
Reconciliation
Observability



Nói đơn giản:

Hệ thống tốt không phải là hệ thống không bao giờ bị retry, duplicate, timeout hay crash. Hệ thống tốt là hệ thống đã chuẩn bị trước cho những chuyện đó và vẫn giữ dữ liệu đúng.

Danh sách thuật ngữ / keyword quan trọng

1. Idempotency

Là khả năng xử lý cùng một request nhiều lần nhưng kết quả cuối cùng vẫn giống như xử lý một lần. Rất quan trọng với payment, transfer, order, refund.

2. Idempotency Key

Một key duy nhất do client hoặc server tạo ra để nhận diện một request nghiệp vụ. Nếu request bị gửi lại với cùng key, server trả lại kết quả cũ thay vì xử lý lại.

3. Request Hash

Giá trị hash của nội dung request. Dùng để phát hiện trường hợp cùng idempotency key nhưng body request khác nhau.

4. Race Condition

Lỗi xảy ra khi nhiều thread/process cùng đọc và ghi một dữ liệu đồng thời, dẫn đến kết quả sai tùy theo thứ tự thực thi.

5. Unique Constraint

Ràng buộc ở database đảm bảo một cột hoặc tổ hợp cột không bị trùng. Đây là lớp bảo vệ rất quan trọng để chống duplicate trong traffic cao.

6. Transaction Boundary

Ranh giới của một transaction. Trong microservice, mỗi service thường chỉ có transaction local trong database của chính nó.

7. Local Transaction

Transaction chỉ diễn ra trong một database/service. Khác với distributed transaction bao trùm nhiều service.

8. Distributed Transaction

Transaction trải rộng trên nhiều service/database. Thường phức tạp, khó scale, dễ lock lâu, nên ít được dùng rộng rãi trong microservice hiện đại.

9. Outbox Pattern

Pattern ghi event vào bảng outbox trong cùng transaction với dữ liệu chính. Sau đó một worker publish event ra message broker. Dùng để tránh mất event khi vừa ghi DB vừa publish message.

10. Dual-Write Problem

Vấn đề xảy ra khi hệ thống phải ghi vào hai nơi khác nhau, ví dụ vừa update database vừa publish message. Một bên thành công, một bên fail sẽ gây mất nhất quán.

11. Saga Pattern

Pattern chia một giao dịch lớn qua nhiều service thành nhiều local transaction nhỏ. Nếu bước sau fail thì dùng hành động bù để hoàn tác logic.

12. Compensating Transaction

Hành động bù khi một bước trong saga fail. Ví dụ đã trừ tiền nhưng giữ hàng thất bại thì cần refund tiền.

13. Choreography Saga

Kiểu saga trong đó các service giao tiếp qua event và tự phản ứng với nhau, không có service trung tâm điều phối.

14. Orchestration Saga

Kiểu saga có một orchestrator trung tâm điều phối các bước của business flow.

15. Idempotent Consumer

Consumer được thiết kế để nếu nhận cùng một message nhiều lần thì chỉ xử lý business logic một lần.

16. Message Deduplication

Cơ chế phát hiện và bỏ qua message đã xử lý trước đó, thường dựa trên `eventId`, `messageId`, hoặc `transactionId`.

17. At-Least-Once Delivery

Cơ chế message broker đảm bảo message được gửi ít nhất một lần. Điều này nghĩa là message có thể bị gửi nhiều lần.

18. Exactly-Once Semantics

Cơ chế đảm bảo xử lý đúng một lần trong một số điều kiện nhất định. Tuy nhiên khi update database bên ngoài message broker, vẫn nên thiết kế idempotency ở tầng business.

19. Optimistic Locking

Cơ chế chống update đè nhau bằng version number. Khi update, hệ thống kiểm tra version cũ còn đúng không. Nếu version đã đổi, update fail và phải retry/read lại.

20. Pessimistic Locking

Cơ chế khóa dữ liệu khi transaction đang xử lý để transaction khác không thể sửa cùng lúc. Đảm bảo chặt nhưng có thể giảm hiệu năng nếu dùng nhiều.

21. Distributed Lock

Lock dùng trên nhiều instance/service, thường triển khai bằng Redis, Zookeeper, etcd. Hữu ích nhưng không nên là lớp bảo vệ duy nhất cho dữ liệu quan trọng.

22. Exponential Backoff

Chiến lược retry với thời gian chờ tăng dần sau mỗi lần lỗi, ví dụ 100ms, 300ms, 1s, 3s.

23. Jitter

Thêm độ ngẫu nhiên vào thời gian retry để tránh nhiều request retry cùng lúc làm hệ thống quá tải.

24. Dead Letter Queue / DLQ

Queue chứa các message xử lý thất bại nhiều lần. Giúp message lỗi không làm nghẽn luồng xử lý chính.

25. State Machine

Mô hình quản lý trạng thái và quy tắc chuyển trạng thái của một nghiệp vụ. Rất hữu ích cho payment, transfer, order, refund.

26. Eventual Consistency

Mô hình nhất quán cuối cùng. Dữ liệu giữa các service có thể lệch tạm thời, nhưng sau một thời gian sẽ đồng bộ.

27. Strong Consistency

Dữ liệu luôn nhất quán ngay tại thời điểm đọc/ghi. Chặt hơn nhưng khó scale hơn trong hệ thống phân tán.

28. CQRS

Command Query Responsibility Segregation. Pattern tách phần ghi dữ liệu và phần đọc dữ liệu để tối ưu riêng từng luồng.

29. Ledger

Sổ ghi nhận giao dịch, thường dùng trong tài chính. Ledger lưu từng debit/credit entry, giúp audit và đối soát.

30. Reconciliation

Quá trình đối soát dữ liệu định kỳ để phát hiện giao dịch lệch, thiếu event, thiếu ledger entry hoặc trạng thái bị kẹt.

31. Correlation ID

ID dùng để trace một request xuyên qua nhiều service. Rất quan trọng khi debug hệ thống microservice.

32. Trace ID

ID dùng trong distributed tracing để theo dõi toàn bộ hành trình request qua các service.

33. Queue Lag

Độ trễ/tồn đọng của message trong queue. Queue lag tăng cao thường là dấu hiệu consumer xử lý không kịp.

34. Backpressure

Cơ chế làm chậm hoặc giới hạn request đầu vào khi hệ thống phía sau đang quá tải.

35. Atomic Operation

Thao tác được thực hiện như một đơn vị không thể chia nhỏ. Hoặc thành công toàn bộ, hoặc không thay đổi gì. Ví dụ update balance có điều kiện trong một câu SQL.



giải thích rõ hơn phần 11. Distributed Lock được không?.
đi từ gốc rễ vấn đề luôn nha

Đã suy nghĩ trong vài giây >

Được. Mình đi từ gốc rễ luôn: **Distributed Lock** sinh ra để giải quyết vấn đề nhiều **instance/service** cùng muốn xử lý một tài nguyên tại cùng một thời điểm.

Nhưng điểm quan trọng là:

Distributed Lock không phải là “cây đũa thần” để đảm bảo dữ liệu đúng. Nó chỉ là một cơ chế phối hợp giữa nhiều process/service. Dữ liệu đúng vẫn phải được bảo vệ cuối cùng bằng database transaction, unique constraint, idempotency, versioning.

1. Gốc rễ vấn đề: tại sao cần lock?

Giả sử hệ thống ban đầu chỉ có 1 server.

```
Client
|
App Server 1
|
Database
```



Nếu chỉ có một server, một số đoạn code xử lý tuần tự hoặc dùng `synchronized`, `ReentrantLock` trong Java là có thể kiểm soát được trong phạm vi process đó.

Ví dụ:

```
synchronized(lock) {
    // xử lý trừ tiền
}
```

Nhưng khi traffic tăng cao, ta scale app thành nhiều instance:

```
Client
|
```



```

Load Balancer
|
|---- App Server 1
|---- App Server 2
|---- App Server 3
|
Database

```

Lúc này synchronized trong Java **không còn đủ**.

Vì synchronized chỉ khóa trong **một JVM/process**.

Nếu request 1 vào App Server 1, request 2 vào App Server 2, thì mỗi server có lock riêng trong memory của nó.

```

App Server 1 có lock riêng
App Server 2 có lock riêng
App Server 3 có lock riêng

```



Nghĩa là 3 server vẫn có thể cùng xử lý một tài nguyên.

Ví dụ khách hàng bấm nút rút tiền 2 lần rất nhanh:

```

Request 1 -> App Server 1
Request 2 -> App Server 2

```



Nếu cả hai cùng xử lý tài khoản A, có thể xảy ra race condition.

Đây là lý do người ta nghĩ đến **Distributed Lock**.

2. Distributed Lock là gì?

Distributed Lock là một cái khóa dùng chung giữa nhiều server/service khác nhau.

Thay vì lock nằm trong memory của từng app server, lock được đặt ở một nơi mà tất cả server đều nhìn thấy được, ví dụ:

```

Redis
Zookeeper
etcd
Database
Consul

```



Mô hình:


```
App Server 1 \
App Server 2 ----> Redis Lock
App Server 3 /
```



Nếu App Server 1 lấy được lock `account:A`, thì App Server 2 và 3 không lấy được lock đó cho đến khi App Server 1 thả lock hoặc lock hết hạn.

Ví dụ:

```
Lock key: wallet:account:A
```



Ý nghĩa:

Trong một thời điểm, chỉ một process được xử lý logic liên quan đến account A.

3. Ví dụ đơn giản: không có distributed lock thì lỗi thế nào?

Giả sử account A có balance = 1,000,000.

Có 2 request rút tiền cùng lúc, mỗi request rút 800,000.

```
Request 1: withdraw 800,000
Request 2: withdraw 800,000
```



Nếu không lock, có thể xảy ra:

```
App 1 đọc balance = 1,000,000
App 2 đọc balance = 1,000,000

App 1 thấy đủ tiền
App 2 cũng thấy đủ tiền

App 1 update balance = 200,000
App 2 update balance = 200,000
```



Kết quả database cuối cùng là 200,000, nhìn tưởng chỉ bị rút 800,000. Nhưng thực tế hệ thống có thể đã tạo 2 giao dịch withdraw thành công. Ledger/log/payment record có thể lệch.

Hoặc nếu update kiểu:

```
UPDATE wallet SET balance = balance - 800000
```

```
WHERE account_id = 'A';
```

thì có thể balance thành -600,000 nếu không check điều kiện đủ tiền.

Distributed lock cố gắng làm như sau:

```
Request 1 lấy lock account:A thành công
Request 2 lấy lock account:A thất bại, phải chờ hoặc trả lỗi

Request 1 xử lý xong, thả lock
Request 2 sau đó mới xử lý tiếp
```

Như vậy 2 request không còn xử lý song song trên cùng một account.

4. Vì sao lock local trong Java không đủ?

Ví dụ dùng Java:

```
private final ReentrantLock lock = new ReentrantLock();

public void withdraw(String accountId, BigDecimal amount) {
    lock.lock();
    try {
        // xử lý rút tiền
    } finally {
        lock.unlock();
    }
}
```

Đoạn này chỉ khóa trong một instance.

Nếu hệ thống có 3 instance:

```
App 1: có ReentrantLock riêng
App 2: có ReentrantLock riêng
App 3: có ReentrantLock riêng
```

Request vào App 1 và App 2 vẫn có thể chạy song song.

Muốn nhiều instance cùng tôn trọng một khóa, lock phải nằm ở nơi chung. Đó là distributed lock.

5. Redis Distributed Lock hoạt động cơ bản như thế nào?

Redis hay được dùng để làm distributed lock vì nó nhanh và hỗ trợ lệnh atomic.

Cách lấy lock cơ bản:

```
SET lock_key unique_token NX PX 30000
```



Ý nghĩa:

```
SET      : set key vào Redis
lock_key : tên khóa
unique_token : giá trị định danh người giữ khóa
NX       : chỉ set nếu key chưa tồn tại
PX 30000 : key tự hết hạn sau 30 giây
```



Ví dụ:

```
SET wallet:lock:account:A request-123 NX PX 30000
```



Nếu Redis trả OK, nghĩa là lấy lock thành công.

Nếu Redis trả null, nghĩa là đã có process khác giữ lock.

Luồng:

```
Request 1:
SET wallet:lock:account:A req-1 NX PX 30000
=> OK
=> xử lý

Request 2:
SET wallet:lock:account:A req-2 NX PX 30000
=> fail
=> không xử lý ngay
```



Sau khi xử lý xong, Request 1 thả lock.

Nhưng thả lock cũng phải làm đúng.

6. Vì sao cần unique token khi lock?

Nhiều người mới làm distributed lock sẽ làm kiểu:

```
SET wallet:lock:account:A true NX PX 30000
```



Sau đó xử lý xong:

```
DEL wallet:lock:account:A
```



Nghe có vẻ ổn, nhưng thực tế rất nguy hiểm.

Xem tình huống này:

T = 0s

Request 1 lấy lock, TTL = 30s



T = 35s

Request 1 xử lý quá lâu, lock tự hết hạn

T = 36s

Request 2 lấy được lock mới

T = 40s

Request 1 xử lý xong và gọi DEL lock

Nếu Request 1 gọi DEL wallet:lock:account:A , nó sẽ xóa lock của Request 2.

Lúc này Request 3 có thể vào và lấy lock, trong khi Request 2 vẫn đang xử lý.

Kết quả là lock mất tác dụng.

Vì vậy khi lấy lock phải lưu token riêng:

```
Request 1 lock value = req-1-token
```

```
Request 2 lock value = req-2-token
```



Khi unlock, chỉ được xóa nếu token đang trong Redis đúng là token của mình.

Logic đúng:

Nếu Redis key value == token của mình:

xóa lock

Ngược lại:

không xóa



Trong Redis phải dùng Lua script để check-and-delete atomic:

```
if redis.call("GET", KEYS[1]) == ARGV[1] then
    return redis.call("DEL", KEYS[1])
else
```

```
return 0  
end
```

Vì nếu làm GET rồi DEL bằng hai lệnh riêng, giữa hai lệnh đó lock có thể đã đổi chủ.

7. Vấn đề TTL: lock phải có hạn sử dụng

Distributed lock cần TTL vì nếu service lấy lock rồi chết, lock sẽ không bao giờ được thả.

Ví dụ:

```
App 1 lấy lock account:A  
App 1 bị crash  
Không ai gọi unlock
```



Nếu lock không có TTL, account A bị khóa mãi mãi.

Do đó phải set TTL:

```
Lock tự hết hạn sau 30 giây
```



Nhưng TTL lại tạo ra vấn đề khác:

Nếu TTL quá ngắn, process chưa xử lý xong thì lock hết hạn, process khác vào xử lý song song.

Nếu TTL quá dài, khi process chết, tài nguyên bị khóa lâu.

Đây là bài toán khó.

Ví dụ:

```
TTL = 10 giây  
Nhưng xử lý thực tế mất 15 giây  
=> lock hết hạn giữa chừng  
=> request khác vào  
=> duplicate processing
```



Cho nên khi dùng Redis lock, phải tính:

```
Thời gian xử lý bình thường là bao lâu?  
Worst-case là bao lâu?  
Có gọi external API không?  
Có thể bị GC pause không?  
Có thể bị DB chậm không?
```



Nếu không chắc, distributed lock rất dễ sai.

8. Watchdog / lock renewal là gì?

Một số thư viện như Redisson có cơ chế **watchdog**.

Ý tưởng:

Process lấy lock với TTL 30 giây.
Nếu process vẫn sống và vẫn đang xử lý,
watchdog tự gia hạn lock thêm.



Ví dụ:

T = 0s: lock TTL 30s
T = 10s: watchdog gia hạn lại TTL 30s
T = 20s: watchdog gia hạn lại TTL 30s
...



Như vậy tránh lock hết hạn khi job chạy lâu.

Nhưng watchdog cũng không phải tuyệt đối an toàn.

Nếu process bị pause lâu, network lag, Redis lag, hoặc JVM stop-the-world GC lâu, watchdog có thể không gia hạn kịp.

Vì vậy ngay cả khi dùng watchdog, vẫn không nên xem distributed lock là lớp bảo vệ dữ liệu cuối cùng.

9. Distributed Lock giải quyết cái gì và không giải quyết cái gì?

Distributed lock giúp:

Giảm khả năng nhiều instance cùng xử lý một resource.
Điều phối công việc giữa nhiều server.
Tránh chạy song song một job quan trọng.
Giảm race condition ở tầng application.



Nhưng distributed lock **không** đảm bảo tuyệt đối:

Không đảm bảo database không bị ghi sai nếu code sai.
Không thay thế unique constraint.



Không thay thế transaction.
Không thay thế idempotency.
Không thay thế optimistic/pessimistic locking ở DB.
Không đảm bảo external API không xử lý trùng.
Không tự rollback nếu service chết giữa chừng.

Nói thẳng theo kinh nghiệm production:

Distributed lock là lớp giảm xung đột. Database transaction và idempotency mới là lớp đảm bảo dữ liệu đúng.

10. Ví dụ thiết kế sai: chỉ dùng Redis lock để chống trùng

Giả sử code:

```
public void withdraw(String accountId, BigDecimal amount) {  
    String lockKey = "wallet:lock:" + accountId;  
  
    if (!redis.tryLock(lockKey)) {  
        throw new RuntimeException("Account is busy");  
    }  
  
    try {  
        Wallet wallet = walletRepository.findByAccountId(accountId);  
  
        if (wallet.getBalance().compareTo(amount) < 0) {  
            throw new RuntimeException("Insufficient balance");  
        }  
  
        wallet.setBalance(wallet.getBalance().subtract(amount));  
        walletRepository.save(wallet);  
  
    } finally {  
        redis.unlock(lockKey);  
    }  
}
```

Nhìn thì ổn. Nhưng vẫn có rủi ro.

Rủi ro 1: Lock hết hạn trước khi xử lý xong.

Request 1 lấy lock
DB bị chậm, xử lý lâu
Lock hết hạn



Request 2 lấy lock
Cả hai cùng update DB

Rủi ro 2: Unlock sai lock nếu không dùng token.

Rủi ro 3: Redis failover làm mất lock.

Rủi ro 4: Một path khác trong code update balance mà không dùng lock.

Rủi ro 5: Service xử lý xong DB nhưng crash trước khi ghi transaction log.

Vì vậy chỉ dùng Redis lock là không đủ.

11. Thiết kế đúng hơn: Redis lock chỉ là phụ, DB vẫn bảo vệ chính

Với nghiệp vụ rút tiền, thiết kế an toàn hơn là:

1. Có idempotency key / transactionId
2. Có unique constraint trên transactionId
3. Update balance bằng atomic SQL
4. Ghi wallet_transaction trong cùng DB transaction
5. Redis lock nếu cần giảm conflict, nhưng không phải lớp bảo vệ duy nhất

Ví dụ:

```
BEGIN;

INSERT INTO wallet_transactions(transaction_id, account_id, amount, type, status)
VALUES (:txId, :accountId, :amount, 'DEBIT', 'PROCESSING');

UPDATE wallets
SET balance = balance - :amount
WHERE account_id = :accountId
  AND balance >= :amount;

-- kiểm tra row affected = 1

UPDATE wallet_transactions
SET status = 'SUCCESS'
WHERE transaction_id = :txId;

COMMIT;
```

Và database có:


```
CREATE UNIQUE INDEX uk_wallet_tx  
ON wallet_transactions(transaction_id);
```

Nếu request bị gửi lại cùng transactionId, insert sẽ bị duplicate key. Code bắt lỗi rồi đọc lại transaction cũ.

Atomic update:

```
UPDATE wallets  
SET balance = balance - :amount  
WHERE account_id = :accountId  
AND balance >= :amount;
```

giúp tránh balance âm.

Vậy Redis lock nếu có chỉ giúp giảm tình trạng nhiều request cùng tranh account đó. Nhưng kể cả Redis lock fail, database vẫn không cho dữ liệu sai dễ dàng.

12. Khi nào nên dùng Distributed Lock?

Distributed lock phù hợp khi cần đảm bảo một tác vụ không chạy song song trên nhiều instance.

Ví dụ thực tế:

12.1. Chạy scheduled job

Hệ thống có 5 instance, mỗi instance đều có scheduler.

Nếu đến 12 giờ đêm, cả 5 instance cùng chạy job đối soát thì có thể xử lý trùng.

Dùng distributed lock:

```
job:daily-reconciliation:2026-06-24
```



Instance nào lấy được lock thì chạy job. Instance khác bỏ qua.

Đây là use case rất hợp lý.

12.2. Xử lý một resource đắt đỏ

Ví dụ generate report cho cùng một user.

Nếu user bấm nhiều lần, không muốn 5 instance cùng generate report.

Lock:

```
report:user:123
```



Instance đầu tiên chạy, instance sau trả:

```
Report is being generated
```



12.3. Tránh nhiều worker xử lý cùng một batch

Ví dụ batch import file:

```
import:file:abc.csv
```



Chỉ một worker được xử lý file đó.

12.4. Điều phối refresh cache

Ví dụ cache hết hạn, rất nhiều request cùng vào và cùng query DB để rebuild cache. Đây gọi là cache stampede.

Dùng lock:

```
cache:rebuild:product:123
```



Chỉ một instance rebuild cache, các instance khác chờ hoặc trả cache cũ.

12.5. Giảm conflict cho hot account / hot inventory

Ví dụ một account hoặc một SKU có traffic rất cao.

Có thể dùng lock theo key:

```
wallet:account:A  
inventory:sku:IPHONE-15
```



Nhưng với money/inventory, vẫn phải có DB transaction/constraint bảo vệ.

13. Khi nào không nên dùng Distributed Lock?

Không nên dùng distributed lock như giải pháp chính cho tất cả race condition.

13.1. Không dùng để thay database transaction

Nếu cần đảm bảo update DB đúng, hãy dùng transaction, isolation, atomic update, unique constraint.

Distributed lock không thay thế được.

13.2. Không dùng lock quá rộng

Ví dụ lock:

```
wallet:all
```



Mỗi lần rút tiền của bất kỳ account nào cũng dùng cùng một lock.

Hệ thống sẽ bị nghẽn vì tất cả giao dịch chạy tuần tự.

Tốt hơn:

```
wallet:account:{accountId}
```



Chỉ khóa theo account liên quan.

13.3. Không giữ lock trong lúc gọi external service lâu

Ví dụ sai:

```
Lấy lock account:A  
Gọi payment gateway  
Đợi 10 giây  
Update DB  
Thả lock
```



Nếu external service chậm, lock bị giữ lâu, throughput giảm, dễ timeout dây chuyền.

Tốt hơn là tách flow hoặc dùng saga.

13.4. Không dùng lock nếu idempotency/unique key giải quyết tốt hơn

Ví dụ chống tạo order trùng. Thay vì lock user, có thể dùng:

```
Idempotency-Key  
Unique(order_request_id)
```



Thường đơn giản và chắc hơn.

14. Lock key phải thiết kế đúng

Distributed lock tốt hay dở phụ thuộc rất nhiều vào lock key.

Ví dụ nghiệp vụ rút tiền tài khoản A:

```
wallet:account:A
```



Nghiệp vụ transfer từ A sang B:

```
wallet:account:A  
wallet:account:B
```



Nhưng nếu cần lock nhiều resource, phải cực kỳ cẩn thận để tránh deadlock.

Ví dụ:

Request 1 transfer A -> B:

```
lock A trước  
lock B sau
```



Request 2 transfer B -> A:

```
lock B trước  
lock A sau
```



Có thể xảy ra:

```
Request 1 giữ lock A, chờ lock B  
Request 2 giữ lock B, chờ lock A
```



Đây là deadlock.

Best practice: nếu phải lock nhiều resource, luôn lock theo thứ tự cố định.

Ví dụ sort accountId:

account A < account B
Luôn lock A trước, B sau



Dù transfer A -> B hay B -> A, vẫn lock theo thứ tự alphabet/id tăng dần.

15. Lock timeout và wait timeout

Khi không lấy được lock, hệ thống phải quyết định làm gì.

Có vài lựa chọn:

Cách 1: Fail fast

Nếu không lấy được lock, trả lỗi ngay:

409 Conflict
Account is being processed, please retry later



Phù hợp với API không cần chờ lâu.

Cách 2: Wait một khoảng ngắn

Ví dụ chờ tối đa 2 giây để lấy lock.

Nếu lấy được thì xử lý.

Nếu không lấy được thì trả lỗi.

Cách 3: Đưa vào queue

Với nghiệp vụ cần xử lý chắc chắn, thay vì client chờ, có thể đưa request vào queue và xử lý tuần tự theo key.

Ví dụ các transaction của cùng account được partition vào cùng Kafka partition.

Cách này thường tốt hơn distributed lock cho throughput lớn.

16. Redis lock và vấn đề crash giữa chừng

Giả sử flow:

1. Lấy lock
2. Trừ tiền trong DB



3. Gửi event
4. Thả lock

Nếu service chết sau bước 2 nhưng trước bước 3:

DB đã trừ tiền
Event chưa gửi
Lock sau đó hết hạn



Distributed lock không giúp khôi phục event bị mất.

Vấn đề này phải xử lý bằng Outbox Pattern.

Thiết kế đúng:

BEGIN TRANSACTION
1. Trừ tiền
2. Ghi wallet_transaction
3. Ghi outbox_event
COMMIT



Sau đó outbox publisher gửi event.

Lock không giải quyết dual-write problem. Outbox mới giải quyết.

17. Redis lock và vấn đề xử lý trùng external API

Ví dụ bạn lấy lock rồi gọi payment gateway:

1. Lấy lock payment:order-123
2. Gọi gateway charge tiền
3. Lưu DB
4. Thả lock



Nếu gateway charge thành công nhưng service timeout/crash trước khi lưu DB thì sao?

Client retry, lock đã hết hạn, hệ thống gọi gateway charge lần nữa.

Khách có thể bị charge 2 lần.

Distributed lock không giải quyết được vấn đề này.

Cách đúng:

Khi gọi external payment gateway, phải dùng idempotency key hoặc merchant transaction ID.

```
gateway_charge(orderId = order-123, idempotencyKey = pay-abc)
```



Nếu gọi lại cùng key, gateway phải trả kết quả cũ, không charge lại.

Bên mình cũng lưu payment attempt với unique key.

18. Distributed lock vs Database lock khác nhau thế nào?

Database lock

Khóa trực tiếp dữ liệu trong database.

Ví dụ:

```
SELECT * FROM wallets WHERE account_id = 'A' FOR UPDATE;
```

Nó đảm bảo trong phạm vi transaction DB, không transaction khác sửa row đó cùng lúc.

Ưu điểm:

Rất sát với dữ liệu.

Mạnh hơn cho tính đúng đắn dữ liệu.

Đi cùng transaction.

Nhược điểm:

Có thể làm DB nghẽn nếu lock lâu.

Không phù hợp để khóa tác vụ không nằm trong DB.

Distributed lock

Khóa ở hệ thống ngoài như Redis.

Ví dụ:

```
SET wallet:lock:A token NX PX 30000
```



Ưu điểm:

Nhanh.

Dùng được cho nhiều loại resource, không chỉ DB row.

Giảm conflict trước khi vào DB.

Hợp với scheduled job, cache rebuild, batch processing.

Nhược điểm:

Không gắn chặt với DB transaction.

Có TTL, crash, failover, network issue.

Không phải lớp bảo vệ cuối cho dữ liệu quan trọng.

19. Distributed lock vs Optimistic Locking

Optimistic Locking

Không khóa trước. Cứ cho nhiều request chạy. Khi update thì kiểm tra version.

```
UPDATE wallet
SET balance = :newBalance,
    version = version + 1
WHERE account_id = :accountId
AND version = :oldVersion;
```

Nếu version không khớp thì update fail.

Phù hợp khi conflict không quá nhiều.

Distributed Lock

Khóa trước khi xử lý.

Ai lấy được lock thì xử lý.
Ai không lấy được thì chờ hoặc fail.



Phù hợp khi muốn tránh nhiều process chạy song song ngay từ đầu.

Trong hệ thống lớn có thể kết hợp:

Redis lock để giảm conflict

+

Optimistic locking/atomic update để đảm bảo DB đúng



20. Distributed lock vs Idempotency

Hai cái này giải quyết hai vấn đề khác nhau.

Distributed Lock

Giải quyết:

Không muốn nhiều process xử lý cùng một resource cùng lúc.



Ví dụ:

Không muốn 2 worker cùng rebuild cache product 123.



Idempotency

Giải quyết:

Cùng một request/message bị gửi lại nhiều lần thì không được tạo nhiều kết quả.



Ví dụ:

Client gửi payment request 3 lần nhưng chỉ charge 1 lần.



Với payment/transfer, idempotency quan trọng hơn distributed lock.

Distributed lock có thể giảm khả năng chạy song song, nhưng idempotency mới giúp an toàn khi retry.

21. Một flow tốt khi dùng Redis lock

Ví dụ xử lý inventory reservation cho SKU ABC .

Request reserve item SKU ABC, quantity 1



Flow tốt:



1. Nhận request với idempotencyKey
2. Kiểm tra idempotency record
3. Lấy Redis lock inventory:sku:ABC với token + TTL
4. Mở DB transaction
5. Insert reservation với unique requestId
6. Atomic update stock:
`available_qty = available_qty - 1`
`WHERE sku = ABC AND available_qty >= 1`
7. Commit DB
8. Lưu response vào idempotency record
9. Unlock bằng token

Nhưng chú ý: nếu Redis lock lỗi, DB vẫn phải bảo vệ:

```
UPDATE inventory
SET available_qty = available_qty - :qty
WHERE sku = :sku
AND available_qty >= :qty;
```

Và reservation có unique key:

```
UNIQUE(reservation_id)
UNIQUE(idempotency_key)
```

22. Code Java minh họa Redis lock đúng hơn

Ví dụ pseudo-code:

```
public ReservationResult reserve(String sku, int qty, String idemKey) {
    IdempotencyRecord old = idempotencyRepository.find(idemKey);
    if (old != null && old.isCompleted()) {
        return old.getResponse();
    }

    String lockKey = "inventory:lock:sku:" + sku;
    String token = UUID.randomUUID().toString();

    boolean locked = redis.setNxPx(lockKey, token, Duration.ofSeconds(30));
    if (!locked) {
        throw new ConflictException("Resource is busy, please retry later");
    }
}
```

```

try {
    return transactionTemplate.execute(status -> {
        try {
            idempotencyRepository.insertProcessing(idemKey);

            reservationRepository.insert(
                new Reservation(idemKey, sku, qty, "PROCESSING")
            );

            int updatedRows = inventoryRepository.decreaseIfEnough(sku, qty);

            if (updatedRows == 0) {
                reservationRepository.markFailed(idemKey);
                idempotencyRepository.markFailed(idemKey, "OUT_OF_STOCK");
                throw new BusinessException("Out of stock");
            }

            reservationRepository.markSuccess(idemKey);
            ReservationResult result = new ReservationResult("SUCCESS");

            idempotencyRepository.markSuccess(idemKey, result);

            return result;
        } catch (DuplicateKeyException e) {
            return idempotencyRepository.find(idemKey).getResponse();
        }
    });
} finally {
    redis.unlockIfTokenMatches(lockKey, token);
}
}

```

Điểm đáng chú ý:

Không unlock mù quáng.

Có idempotency.

Có DB transaction.

Có unique key.

Có atomic update.

Lock chỉ là một lớp hỗ trợ.

23. Redlock là gì? Có nên dùng không?

Redlock là thuật toán distributed lock dùng nhiều Redis node độc lập. Ý tưởng là client phải lấy được lock trên đa số node thì mới xem là thành công.

Ví dụ có 5 Redis node, client cần lock thành công ít nhất 3 node.

Mục tiêu là giảm rủi ro khi một Redis node fail.

Tuy nhiên Redlock từng gây nhiều tranh luận trong cộng đồng kỹ thuật, vì distributed locking trong môi trường network partition, clock drift, process pause là vấn đề khó.

Trong thực tế doanh nghiệp:

Nếu dùng lock cho job/cache/task không quá critical, Redis lock hoặc Redisson thường đủ.

Nếu dùng cho dữ liệu tài chính quan trọng, đừng chỉ dựa vào Redlock. Vẫn cần DB transaction, idempotency, unique constraint, ledger, reconciliation.

Nói dễ hiểu:

Redlock có thể làm distributed lock "tốt hơn", nhưng không biến distributed lock thành bảo đảm tuyệt đối cho business correctness.

24. Ví dụ production: scheduled job chạy trùng

Giả sử service có 4 instance:

```
Settlement Service instance 1
Settlement Service instance 2
Settlement Service instance 3
Settlement Service instance 4
```



Mỗi ngày 00:00 chạy job settlement.

Nếu không có lock:

```
Cả 4 instance cùng chạy settlement ngày 2026-06-24
Có thể tạo file settlement 4 lần
Có thể gửi payout 4 lần
Có thể update status trùng
```



Dùng distributed lock:

```
lock key = settlement:daily:2026-06-24
```



Instance 1 lấy được lock:

```
SET settlement:daily:2026-06-24 token-1 NX PX 3600000
```



Instance khác không lấy được lock thì bỏ qua.

Nhưng vẫn phải có database bảo vệ:

```
settlement_date UNIQUE  
payout_batch_id UNIQUE  
transaction status transition hợp lệ
```



Vì nếu lock lỗi hoặc job bị chạy lại thủ công, database vẫn không được tạo settlement trùng.

25. Ví dụ production: cache stampede

Một product P123 có cache hết hạn.

10,000 request cùng vào.

Nếu không có lock:

```
10,000 request cùng query database  
10,000 request cùng rebuild cache  
Database bị spike
```



Dùng distributed lock:

```
cache:rebuild:product:P123
```



Request đầu tiên lấy lock và rebuild cache.

Các request khác có thể:

```
Trả cache cũ nếu có  
Hoặc chờ ngắn  
Hoặc trả response degraded
```



Đây là use case rất tốt cho distributed lock vì nếu lock fail thì dữ liệu không sai nghiêm trọng, chỉ có thể tốn tài nguyên hơn.

26. Tóm tắt tư duy đúng về Distributed Lock

Khi nghe "Distributed Lock", đừng hiểu là:

"Có lock rồi thì dữ liệu chắc chắn đúng."

Nên hiểu là:

"Lock giúp nhiều instance phối hợp để giảm khả năng cùng xử lý một resource. Nhưng dữ liệu đúng vẫn phải được enforce bởi database transaction, idempotency và constraint."

Công thức thực tế:

```
Business critical operation
= Idempotency
+ Database transaction
+ Unique constraint
+ Atomic update / optimistic locking / pessimistic locking
+ Outbox nếu có event
+ Reconciliation
+ Distributed lock nếu cần giảm conflict
```



Distributed lock nằm ở gần cuối, không phải đầu tiên.

27. Khi interview senior nên trả lời thế nào?

Có thể trả lời như sau:

Trong hệ thống microservice scale nhiều instance, lock trong memory như `synchronized` hoặc `ReentrantLock` không đủ vì mỗi instance có memory riêng. Khi nhiều request cùng xử lý một tài nguyên, ví dụ cùng account hoặc cùng order, ta có thể dùng distributed lock như Redis/Zookeeper/etcd để đảm bảo tại một thời điểm chỉ một instance được xử lý resource đó.

Tuy nhiên, tôi sẽ không xem distributed lock là cơ chế đảm bảo dữ liệu đúng tuyệt đối. Distributed lock có nhiều edge case như TTL hết hạn khi process chưa xử lý xong, service crash, network partition, Redis failover, unlock nhầm lock của process khác. Vì vậy nếu dùng Redis lock, bắt buộc phải dùng `SET key value NX PX ttl`, value phải là unique token, và khi unlock phải check token bằng Lua script để tránh xóa nhầm lock của request khác.

Trong nghiệp vụ critical như payment, wallet, inventory, distributed lock chỉ nên là lớp giảm conflict. Lớp bảo vệ chính vẫn là database transaction, unique constraint,

idempotency key, atomic update hoặc optimistic/pessimistic locking. Ví dụ khi trừ tiền, tôi vẫn dùng câu SQL `atomic UPDATE wallet SET balance = balance - amount WHERE account_id = ? AND balance >= amount`, kèm transaction log có unique transactionId. Như vậy kể cả lock bị expire hoặc request bị retry, database vẫn không bị update sai hoặc xử lý trùng.

Distributed lock rất hữu ích cho các use case như scheduled job chạy trên nhiều instance, cache rebuild để tránh cache stampede, batch processing, hoặc giảm conflict trên hot resource. Nhưng với business transaction quan trọng, tôi luôn kết hợp lock với idempotency, transaction, outbox và reconciliation.

Danh sách thuật ngữ / keyword quan trọng

1. Distributed Lock

Cơ chế khóa dùng chung giữa nhiều server/service để đảm bảo tại một thời điểm chỉ một process xử lý một resource nhất định.

2. Local Lock

Lock nằm trong memory của một process/JVM, ví dụ `synchronized`, `ReentrantLock`. Không có tác dụng giữa nhiều instance khác nhau.

3. Redis Lock

Distributed lock triển khai bằng Redis, thường dùng lệnh `SET key value NX PX ttl`.

4. NX

Option của Redis nghĩa là chỉ set key nếu key chưa tồn tại. Dùng để đảm bảo chỉ một process lấy được lock.

5. PX

Option của Redis để set thời gian sống của key theo milliseconds. Dùng để lock tự hết hạn nếu process chết.

6. TTL

Time To Live. Thời gian lock tồn tại trước khi tự expire.

7. Unique Token

Giá trị duy nhất đại diện cho owner của lock. Dùng để đảm bảo chỉ process giữ lock mới được unlock.

8. Lua Script

Script chạy atomic trong Redis. Thường dùng để check token rồi delete lock trong một thao tác không bị chen ngang.

9. Watchdog

Cơ chế tự gia hạn lock nếu process vẫn đang xử lý. Redisson có hỗ trợ watchdog.

10. Deadlock

Tình trạng hai process giữ lock của nhau và cùng chờ nhau mãi. Hay xảy ra khi lock nhiều resource nhưng không có thứ tự cố định.

11. Fail Fast

Không chờ lock quá lâu. Nếu không lấy được lock thì trả lỗi ngay để client retry sau.

12. Cache Stampede

Hiện tượng cache hết hạn và rất nhiều request cùng query database để rebuild cache, làm database bị spike.

13. Redlock

Thuật toán distributed lock dùng nhiều Redis node và cần đạt đa số lock thành công. Có ích nhưng vẫn không thay thế được transaction/idempotency.

14. Atomic Update

Câu update thực hiện kiểm tra điều kiện và thay đổi dữ liệu trong một thao tác atomic ở database.

15. Fencing Token

Một số hệ thống dùng token tăng dần để ngăn process cũ ghi dữ liệu sau khi lock đã hết hạn và process mới đã lấy lock.

16. Idempotency

Cùng một request xử lý nhiều lần nhưng kết quả cuối cùng vẫn như xử lý một lần. Quan trọng hơn distributed lock trong payment/transfer.

17. Unique Constraint

Ràng buộc database chống dữ liệu trùng. Đây là lớp bảo vệ cuối rất quan trọng.

18. Reconciliation

Job đối soát để phát hiện dữ liệu lệch, giao dịch kẹt, event thiếu hoặc xử lý trùng.



↳ 13.3. Không giữ lock trong lúc gọi external service lâu -----
 ----- Ví dụ sai: Lấy lock account:AGọi payment gatewayĐợi 10 giâyUpdate
 DBThả lock Nếu external service chậm, lock bị giữ lâu, throughput giảm, dễ timeout đây...

dùng saga như thế nào trong trường hợp này?

Đã suy nghĩ trong 1 giây >

Đúng rồi, câu hỏi này rất hay. Mình sẽ lấy đúng case này:

Sai:

1. Lấy lock account:A
2. Gọi payment gateway bên ngoài
3. Đợi 10 giây
4. Update DB
5. Thả lock



Vấn đề là **đang giữ lock trong lúc gọi external service**. External service có thể chậm, timeout, hoặc trả kết quả không rõ ràng. Nếu giữ lock suốt thời gian đó, hệ thống dễ bị nghẽn.

Cách tốt hơn là dùng **Saga** để tách flow thành nhiều bước nhỏ, mỗi bước có transaction riêng, không giữ lock lâu.

1. Gốc rễ vấn đề

Giả sử user nạp tiền / thanh toán / chuyển tiền qua payment gateway.

Flow đơn giản ban đầu có thể là:

```
User request payment
  |
Payment Service
  |
Lấy lock account:A
  |
Gọi payment gateway
  |
Update balance
```



|
Thả lock

Nhìn thì có vẻ an toàn vì có lock. Nhưng nó sai ở điểm: **payment gateway là hệ thống bên ngoài**, mình không kiểm soát được tốc độ và độ ổn định của nó.

Payment gateway có thể:

Trả response sau 10 giây
Timeout nhưng thật ra đã charge thành công
Charge thành công nhưng response bị mất
Trả lỗi tạm thời
Service của mình crash sau khi gateway charge xong



Nếu mình giữ lock trong suốt thời gian chờ gateway, account đó bị khóa rất lâu.

Ví dụ account A đang có nhiều giao dịch:

Request 1 giữ lock account:A và chờ gateway 10 giây
Request 2 muốn xử lý account:A phải chờ
Request 3 muốn xử lý account:A phải chờ
Request 4 muốn xử lý account:A phải chờ



Traffic cao lên là queue chờ lock tăng, thread bị giữ, connection bị giữ, latency tăng, rồi timeout dây chuyền.

Saga giải quyết bằng cách:

Không xử lý một flow dài bằng một transaction/lock dài. Thay vào đó, chia nó thành nhiều bước nhỏ, mỗi bước commit trạng thái rõ ràng, rồi bước sau tiếp tục dựa trên event hoặc orchestrator.

2. Ý tưởng Saga trong case gọi payment gateway

Thay vì làm một flow synchronous dài:

Lock -> gọi gateway -> update DB -> unlock



Ta làm thành nhiều bước:

1. Tạo payment intent / transaction ở trạng thái INITIATED
2. Commit DB ngay
3. Gọi payment gateway bên ngoài mà không giữ DB lock / distributed lock lâu
4. Gateway trả kết quả hoặc callback webhook



5. Update trạng thái payment
6. Nếu payment success thì update wallet/ledger bằng local transaction ngắn
7. Nếu bước sau fail thì dùng compensation/retry/reconciliation

Nói cách khác, Saga biến flow thành một **state machine**.

Ví dụ trạng thái:

```

CREATED
PAYMENT_PENDING
PAYMENT_AUTHORIZED
PAYMENT_FAILED
WALLET_CREDIT_PENDING
WALLET_CREDITED
LEDGER_PENDING
COMPLETED
COMPENSATION_REQUIRED
REFUND_PENDING
REFUNDED

```



Mỗi bước chỉ làm một phần nhỏ, có thể retry an toàn.

3. Ví dụ cụ thể: user nạp tiền vào ví qua payment gateway

Giả sử user muốn nạp 1,000,000 VND vào wallet A.

Sai lầm là:

```

Lấy lock wallet:A
Gọi payment gateway
Nếu success thì cộng balance
Thả lock

```



Thiết kế Saga tốt hơn:

```

Client
|
Payment Service
|
Payment Gateway
|
Wallet Service
|
Ledger Service

```



Flow tổng quát:

1. Payment Service tạo payment transaction PENDING
2. Payment Service gọi gateway với idempotency key
3. Gateway xử lý và trả kết quả hoặc gửi webhook
4. Payment Service nhận kết quả success
5. Payment Service phát event PaymentSucceeded
6. Wallet Service consume event và cộng tiền vào ví bằng local transaction ngắn
7. Ledger Service ghi ledger bằng local transaction ngắn
8. Payment Service/Orchestrator đánh dấu flow COMPLETED

Quan trọng: **không lock wallet trong lúc gọi gateway.**

Wallet chỉ bị lock hoặc update trong bước cộng tiền, và bước đó rất ngắn.

4. Thiết kế theo Orchestration Saga

Trong case payment/banking, mình thường thích **Orchestration Saga** hơn Choreography, vì flow tài chính cần trạng thái rõ ràng, dễ audit, dễ retry.

Sẽ có một service điều phối, ví dụ:

Payment Orchestrator

Nó quản lý flow:

```
CREATE_PAYMENT
CALL_GATEWAY
CONFIRM_PAYMENT
CREDIT_WALLET
WRITE_LEDGER
COMPLETE
```

Luồng chi tiết:

```
Client gửi request nạp tiền
|
Payment Orchestrator tạo payment record
|
Commit DB
|
Gọi payment gateway với idempotency key
|
Nhận kết quả hoặc webhook
|
Nếu success: gửi command CreditWallet
|
Wallet Service cộng tiền bằng transaction ngắn
```

```
|
Ledger Service ghi ledger
|
Payment Orchestrator mark COMPLETED
```

5. Bước 1: tạo transaction nội bộ trước

Khi client gửi request:

```
POST /topups
Idempotency-Key: topup-abc-123
```

Body:

```
{
  "walletId": "A",
  "amount": 1000000
}
```

Payment Service không gọi gateway ngay trong một transaction dài.

Nó làm:

BEGIN TRANSACTION

1. Check idempotency key
2. Tạo payment transaction:
paymentId = pay-001
walletId = A
amount = 1000000
status = CREATED
3. Lưu idempotency record
4. Lưu outbox event/command: CallGatewayRequested

COMMIT

Database:

payments

```
payment_id | wallet_id | amount | status  
pay-001    | A           | 1000000 | CREATED
```

Idempotency:

```
idempotency_key | payment_id | status  
topup-abc-123   | pay-001    | PROCESSING
```



Lúc này transaction DB rất ngắn. Không lock wallet. Không gọi external gateway trong transaction này.

6. Bước 2: gọi payment gateway ngoài transaction chính

Một worker hoặc orchestrator đọc command `CallGatewayRequested`.

Nó update trạng thái:

```
pay-001: CREATED -> PAYMENT_PENDING
```



Sau đó gọi gateway:

```
Gateway charge:  
merchantTransactionId = pay-001  
idempotencyKey = pay-001 hoặc gateway-key-001  
amount = 1000000
```



Điểm cực kỳ quan trọng:

Khi gọi payment gateway, phải truyền **merchant transaction ID** hoặc **idempotency key** sang gateway.

Vì nếu bên mình timeout và gọi lại, gateway phải hiểu đây là cùng một giao dịch, không charge lần 2.

Không được mỗi lần retry lại tạo một transaction mới bên gateway.

Sai:

```
Retry 1: gatewayTransactionId = random-001  
Retry 2: gatewayTransactionId = random-002  
Retry 3: gatewayTransactionId = random-003
```



Đúng:

```
Retry 1: merchantTransactionId = pay-001
Retry 2: merchantTransactionId = pay-001
Retry 3: merchantTransactionId = pay-001
```



Như vậy gateway có thể trả cùng một kết quả cho cùng một giao dịch.

7. Bước 3: xử lý kết quả gateway

Có hai kiểu nhận kết quả.

Kiểu 1: synchronous response

Gateway trả ngay:

```
{
  "status": "SUCCESS",
  "gatewayTransactionId": "gw-999"
}
```

Payment Service update:

```
pay-001: PAYMENT_PENDING -> PAYMENT_SUCCEEDED
```



Rồi phát event:

```
PaymentSucceeded(pay-001)
```



Kiểu 2: webhook/callback

Gateway không trả kết quả ngay, hoặc trả pending. Sau đó gateway gọi webhook:

```
POST /payment-webhook
```

Body:

```
{
  "merchantTransactionId": "pay-001",
  "gatewayTransactionId": "gw-999",
}
```

```
"status": "SUCCESS"
}
```

Payment Service nhận webhook và update trạng thái.

Trong hệ thống payment thực tế, webhook rất quan trọng vì response synchronous có thể timeout hoặc bị mất. Webhook giúp hệ thống nhận kết quả cuối cùng từ gateway.

8. Bước 4: Wallet Service cộng tiền bằng transaction ngân

Khi Payment Service biết payment success, nó phát event:

```
{
  "eventId": "evt-001",
  "eventType": "PaymentSucceeded",
  "paymentId": "pay-001",
  "walletId": "A",
  "amount": 1000000
}
```

Wallet Service consume event.

Wallet không cần giữ lock lâu. Nó chỉ mở transaction ngân:

BEGIN TRANSACTION

1. Check eventId đã xử lý chưa
2. Check paymentId đã credit chưa
3. Cộng balance:
balance = balance + 1000000
4. Ghi wallet_transaction:
paymentId = pay-001
type = CREDIT
amount = 1000000
5. Ghi processed_event
6. Ghi outbox event WalletCredited

COMMIT

Cần unique constraint:

</> SQL



Ví dụ:

```
payment_id | wallet_id | type | amount
pay-001    | A         | CREDIT | 1000000
```



Nếu PaymentSucceeded bị gửi lại 3 lần, Wallet Service vẫn chỉ credit 1 lần vì payment_id + type là unique.

Đây là điểm mấu chốt:

Saga không chống duplicate bằng lock dài. Saga chống duplicate bằng idempotency, unique constraint, processed event, và trạng thái rõ ràng ở từng bước.

9. Bước 5: Ledger Service ghi sổ

Ledger Service cũng consume event hoặc nhận command.

Nó ghi:

```
BEGIN TRANSACTION
```



1. Check paymentId đã ghi ledger chưa
2. Insert ledger entry
3. Mark event processed

```
COMMIT
```

Ledger entry:

```
ledger_entries
```



```
-----
entry_id | payment_id | account | direction | amount
led-001  | pay-001    | A       | CREDIT    | 1000000
```

Unique constraint:

```
UNIQUE(payment_id, direction)
```

Nếu event bị consume lại, ledger không bị ghi trùng.

10. Bước 6: hoàn tất Saga

Sau khi Wallet Service và Ledger Service xong, Orchestrator hoặc Payment Service update:

```
pay-001: COMPLETED
```



Idempotency record cũng được update:

```
topup-abc-123: SUCCESS
response = { paymentId: pay-001, status: COMPLETED }
```



Nếu client retry cùng idempotency key:

```
Idempotency-Key: topup-abc-123
```



Hệ thống trả lại kết quả cũ, không gọi gateway lại, không cộng wallet lại.

11. Vậy Saga thay thế lock dài như thế nào?

Trước đây, flow sai là:

```
Lock wallet A
|
Gọi gateway 10 giây
|
Update wallet
|
Unlock
```



Saga đổi thành:

```
Tạo payment record
|
Commit
|
Gọi gateway không giữ wallet lock
|
Nhận success
|
Mở transaction ngắn ở Wallet Service
|
Credit wallet
|
Commit
```



Điểm khác nhau:

Không giữ lock trong lúc gọi gateway.
Không giữ database transaction trong lúc gọi gateway.
Không giữ row lock trong lúc đợi external service.
Mỗi bước có trạng thái riêng.
Mỗi bước retry được.
Mỗi bước idempotent.



Wallet chỉ bị update ở bước rất ngắn:

```
BEGIN
credit wallet
insert wallet_transaction
COMMIT
```



Thời gian có thể chỉ vài milliseconds đến vài chục milliseconds, thay vì giữ lock 10 giây.

12. Nếu gateway timeout thì Saga xử lý sao?

Đây là case thực tế nhất.

Payment Service gọi gateway:

pay-001



Gateway không trả response sau 10 giây. Bên mình bị timeout.

Sai lầm là mark payment failed ngay.

Vì timeout không có nghĩa là gateway chưa charge.

Có thể gateway đã charge thành công nhưng response bị mất.

Cách xử lý đúng:

pay-001: PAYMENT_PENDING hoặc UNKNOWN



Sau đó hệ thống:

1. Chờ webhook từ gateway
2. Hoặc chạy job query status gateway bằng merchantTransactionId = pay-001
3. Nếu gateway nói success -> tiếp tục saga
4. Nếu gateway nói failed -> mark failed
5. Nếu gateway vẫn unknown -> retry query sau



Trạng thái nên có:

PAYMENT_PENDING
PAYMENT_UNKNOWN
PAYMENT_SUCCEEDED
PAYMENT_FAILED



Không nên:

Timeout = failed ngay



Trong payment, timeout thường phải coi là UNKNOWN/PENDING .

13. Nếu wallet credit fail sau khi gateway charge success thì sao?

Ví dụ:

Gateway charge thành công
Payment Service nhận PaymentSucceeded
Wallet Service đang down



Lúc này tiền đã charge, nhưng ví chưa cộng.

Nếu dùng flow synchronous có thể lỗi lung tung. Nhưng với Saga, trạng thái rõ:

pay-001: PAYMENT_SUCCEEDED
wallet_credit_status: PENDING



Event PaymentSucceeded nằm trong queue/outbox, Wallet Service down thì sau khi lên lại sẽ xử lý tiếp.

Nếu Wallet Service xử lý fail tạm thời, retry.

Nếu fail nhiều lần, đưa vào DLQ và alert.

Nếu không thể credit wallet sau một khoảng thời gian, có thể chuyển sang compensation:

Refund payment



Tức là:

```
Gateway charge success
Wallet credit fail nghiêm trọng
=> tạo RefundRequested
=> gọi gateway refund
=> pay-001 status = REFUNDED
```



Đây chính là **compensating transaction** trong Saga.

14. Compensation trong case này là gì?

Trong database transaction truyền thống, fail thì rollback.

Nhưng với external gateway, không rollback được kiểu DB.

Nếu đã charge tiền khách thành công, muốn "rollback" thì phải gọi nghiệp vụ ngược lại:

```
Refund
```



Đó là compensation.

Ví dụ:

```
Bước 1: Charge payment thành công
Bước 2: Credit wallet thất bại không thể recover
Bước 3: Compensation = Refund payment
```



Flow:

```
PaymentSucceeded
|
CreditWalletFailed
|
RefundRequested
|
Call Gateway Refund
|
RefundSucceeded
|
Payment status = REFUNDED
```



Trạng thái:

```
PAYMENT_SUCCEEDED
CREDIT_FAILED
```



REFUND_PENDING
REFUNDED

Nhưng cần nhớ: compensation cũng có thể fail hoặc timeout. Nên refund cũng phải idempotent, có refundId, có trạng thái, có retry, có reconciliation.

15. Có nên dùng distributed lock trong Saga không?

Có thể dùng, nhưng dùng rất ngán và đúng chỗ.

Ví dụ ở bước Wallet Service credit balance, có thể dùng một trong các cách:

Cách 1: Không dùng Redis lock, chỉ dùng DB atomic/unique

Với credit tiền vào ví, có thể chỉ cần:

```
UPDATE wallets
SET balance = balance + :amount
WHERE wallet_id = :walletId;
```

Kèm unique constraint:

```
UNIQUE(payment_id, type)
```

Vì cộng tiền là atomic và duplicate đã được chặn bởi unique key.

Cách 2: Dùng DB lock ngán

```
SELECT * FROM wallets
WHERE wallet_id = :walletId
FOR UPDATE;
```

Sau đó update balance và insert transaction log trong cùng transaction.

Lock chỉ giữ trong DB transaction ngán.

Cách 3: Dùng Redis lock ngán để giảm conflict

```
lock wallet:A
BEGIN
```



```
credit wallet
insert wallet_transaction
COMMIT
unlock wallet:A
```

Nhưng tuyệt đối không:

```
lock wallet:A
call gateway
wait 10s
credit wallet
unlock
```



Nếu dùng lock trong Saga thì lock chỉ dùng ở **local transaction ngắn**, không bao trùm external call.

16. Orchestration Saga pseudo-flow

Ví dụ orchestrator điều phối top-up:

```
START_TOPUP
|
CREATE_PAYMENT_RECORD
|
CALL_GATEWAY
|
Nếu gateway success:
  CREDIT_WALLET
Nếu gateway failed:
  MARK_PAYMENT_FAILED
Nếu gateway timeout:
  MARK_PAYMENT_UNKNOWN
  WAIT_FOR_WEBHOOK_OR_STATUS_CHECK
```



Sau khi credit wallet:

```
Nếu credit success:
  WRITE_LEDGER
Nếu credit fail temporary:
  RETRY_CREDIT
Nếu credit fail permanent:
  REFUND_PAYMENT
```



Sau khi ledger:

Nếu ledger success:
COMPLETE

Nếu ledger fail:
RETRY_LEDGER hoặc ALERT



Có thể biểu diễn như này:

```

CREATED
|
v
PAYMENT_PENDING
|
|-- gateway success --> PAYMENT_SUCCEEDED
|
|                               |
|                               v
|                               WALLET_CREDIT_PENDING
|                               |
|                               |-- success --> WALLET_CREDITED
|                               |
|                               |
|                               v
|                               LEDGER_PENDING
|                               |
|                               v
|                               COMPLETED
|
|-- gateway failed --> PAYMENT_FAILED
|
|-- gateway timeout --> PAYMENT_UNKNOWN
|
|
v
WAIT WEBHOOK / QUERY STATUS
  
```



17. Choreography Saga thì làm sao?

Nếu không dùng orchestrator, các service tự nghe event.

Flow:

```

TopupCreated
-> Payment Service gọi gateway

PaymentSucceeded
-> Wallet Service credit wallet

WalletCredited
-> Ledger Service ghi ledger
  
```



LedgerWritten

-> Topup Service mark completed

WalletCreditFailed

-> Payment Service refund

Ưu điểm:

Ít service trung tâm.

Service tách biệt hơn.

Nhược điểm:

Flow tài chính phức tạp sẽ khó debug.

Muốn biết `pay-001` đang ở đâu phải trace event qua nhiều service.

Dễ bị event spaghetti.

Với banking/payment, mình thường chọn **Orchestration Saga** nếu flow quan trọng và nhiều bước.

18. Điểm cực kỳ quan trọng: Outbox phải đi chung với Saga

Saga dựa nhiều vào event/command. Nếu update DB xong mà event mất thì Saga bị kẹt.

Ví dụ:

Payment Service update `pay-001` = `PAYMENT_SUCCEEDED`

Nhưng publish `PaymentSucceeded` fail

Wallet Service không bao giờ credit tiền



Vì vậy nên dùng Outbox:

BEGIN TRANSACTION

1. Update payment status = `PAYMENT_SUCCEEDED`

2. Insert outbox event `PaymentSucceeded`

COMMIT



Sau đó outbox publisher gửi event.

Nếu publish fail, event vẫn còn trong outbox để retry.

Không nên:

```
Update DB success  
Publish Kafka fail  
Thôi bỏ qua
```



Với Saga, Outbox gần như là bắt buộc trong flow quan trọng.

19. Inbox / Processed Event cũng quan trọng

Service nhận event cũng phải chống duplicate.

Wallet Service nhận PaymentSucceeded .

```
BEGIN TRANSACTION
```



1. Insert processed_event(eventId)
2. Insert wallet_transaction(paymentId)
3. Update balance
4. Insert outbox WalletCredited

```
COMMIT
```

Nếu event bị gửi lại, bước insert processed_event hoặc wallet_transaction bị duplicate, service biết đã xử lý rồi và bỏ qua.

Đây gọi là **Inbox Pattern** hoặc **Idempotent Consumer**.

Outbox đảm bảo event không mất ở bên gửi.

Inbox đảm bảo event trùng không làm sai ở bên nhận.

20. Nếu client cần response ngay thì sao?

Saga thường là async. Nhưng client có thể muốn biết kết quả ngay.

Có vài cách.

Cách 1: trả về PENDING

Client gọi top-up:

```
{  
  "paymentId": "pay-001",  
}
```

```
"status": "PAYMENT_PENDING"
}
```

Client poll:

```
GET /topups/pay-001
```

Hoặc dùng websocket/push notification.

Đây là cách phổ biến với payment.

Cách 2: cố gắng xử lý sync trong thời gian ngắn, nhưng không giữ lock

Payment Service có thể gọi gateway sync, nhưng vẫn không giữ wallet lock/DB transaction trong lúc gọi.

Flow:

1. Tạo payment CREATED, commit
2. Gọi gateway sync
3. Nếu success, update payment success + outbox
4. Trả response PAYMENT_SUCCEEDED hoặc PROCESSING



Sau đó wallet credit vẫn xử lý qua event hoặc command.

Client có thể thấy:

```
Payment success, wallet updating
```



Hoặc hệ thống chờ thêm một chút để wallet hoàn tất nếu SLA cho phép.

Cách 3: dùng webhook/callback về client

Khi completed thì notify.

21. So sánh flow sai và flow Saga

Flow sai

```
API request
|
lock wallet
|
call gateway 10s
```



```
|  
update wallet  
|  
unlock  
|  
return
```

Vấn đề:

```
Giữ lock lâu  
Giữ thread lâu  
Timeout khó xử lý  
Crash giữa chừng khó recover  
Gateway success nhưng DB fail gây lệch  
Retry dễ charge trùng
```



Flow Saga

```
API request  
|  
create payment record  
commit  
|  
call gateway with idempotency key  
|  
gateway success/webhook  
|  
publish PaymentSucceeded via outbox  
|  
wallet credit in short transaction  
|  
ledger write  
|  
complete
```



Ưu điểm:

```
Không giữ lock lâu  
Mỗi bước có trạng thái  
Retry an toàn  
Duplicate được chặn  
Có compensation nếu bước sau fail  
Có thể recover sau crash  
Dễ audit  
Dễ reconciliation
```



22. Một ví dụ database schema đơn giản

payments

```
payment_id  
wallet_id  
amount  
status  
gateway_transaction_id  
created_at  
updated_at
```



idempotency_records

```
idempotency_key  
request_hash  
payment_id  
status  
response_body  
created_at
```



wallet_transactions

```
wallet_tx_id  
payment_id  
wallet_id  
type  
amount  
status  
created_at
```



Unique:

```
UNIQUE(payment_id, type)
```



outbox_events

```
event_id  
aggregate_id  
event_type  
payload  
status  
created_at  
published_at
```



processed_events

```
event_id  
consumer_name  
processed_at
```



Unique:

```
UNIQUE(event_id, consumer_name)
```



23. Best practice khi dùng Saga cho external payment

Thứ nhất, **luôn tạo transaction nội bộ trước khi gọi gateway.**

Không để gateway là nơi duy nhất biết giao dịch.

Thứ hai, **gọi gateway bằng idempotency key / merchant transaction ID cố định.**

Retry phải dùng cùng key.

Thứ ba, **timeout không được xem là failed ngay.**

Timeout nên là UNKNOWN hoặc PENDING , sau đó chờ webhook hoặc query status.

Thứ tư, **không giữ DB transaction hoặc distributed lock khi gọi gateway.**

External call phải nằm ngoài transaction/lock dài.

Thứ năm, **update trạng thái và publish event bằng Outbox Pattern.**

Tránh DB success nhưng event mất.

Thứ sáu, **consumer phải idempotent.**

Wallet/Ledger phải chống event trùng bằng unique constraint hoặc processed event.

Thứ bảy, **có compensation.**

Nếu payment success nhưng không thể hoàn tất bước sau, cần refund hoặc manual review.

Thứ tám, **có reconciliation job.**

Check các payment stuck ở PAYMENT_PENDING , PAYMENT_UNKNOWN ,
WALLET_CREDIT_PENDING , REFUND_PENDING .

24. Câu trả lời kiểu senior interview

Trong case phải gọi external payment gateway, tôi sẽ không giữ distributed lock hoặc DB transaction trong suốt thời gian gọi gateway, vì external service có thể chậm hoặc timeout, làm lock bị giữ lâu và gây nghẽn hệ thống. Thay vào đó, tôi sẽ thiết kế flow theo Saga.

Đầu tiên, Payment Service tạo một payment transaction nội bộ với trạng thái `CREATED` hoặc `PAYMENT_PENDING`, lưu idempotency key và commit ngay. Sau đó mới gọi payment gateway bằng một `merchantTransactionId` hoặc `idempotencyKey` cố định, ví dụ chính là `paymentId`, để nếu retry thì gateway không charge trùng.

Nếu gateway trả success hoặc gửi webhook success, Payment Service update trạng thái payment thành `PAYMENT_SUCCEEDED` và ghi event `PaymentSucceeded` vào outbox trong cùng transaction. Wallet Service consume event này và cộng tiền vào ví trong một local transaction ngắn, kèm unique constraint theo `paymentId` để đảm bảo event bị gửi lại cũng không credit trùng. Ledger Service cũng ghi ledger theo cách idempotent tương tự.

Nếu gateway timeout, tôi không mark failed ngay mà chuyển payment sang `PAYMENT_UNKNOWN` hoặc giữ `PAYMENT_PENDING`, sau đó chờ webhook hoặc dùng job query lại trạng thái gateway bằng `paymentId`. Nếu payment đã success nhưng bước credit wallet fail tạm thời thì retry. Nếu fail không thể recover, Saga sẽ chạy compensating transaction, ví dụ refund lại tiền cho khách.

Như vậy Saga giúp tách một business transaction dài thành nhiều local transaction ngắn, không giữ lock lâu, có trạng thái rõ ràng, retry được, chống duplicate bằng idempotency/unique constraint, và có compensation khi một bước sau bị lỗi.

Keyword cần nhớ

Saga

Pattern chia một business transaction lớn qua nhiều service thành nhiều local transaction nhỏ. Mỗi bước commit riêng, nếu lỗi thì retry hoặc chạy bước bù.

Orchestration Saga

Có một orchestrator trung tâm điều phối các bước. Phù hợp payment/banking vì dễ audit, dễ quản lý trạng thái.

Choreography Saga

Các service tự nghe event và phản ứng. Phù hợp flow đơn giản nhưng dễ khó debug khi flow phức tạp.

Compensation

Hành động bù khi một bước đã thành công nhưng bước sau fail. Ví dụ payment charge thành công nhưng credit wallet fail thì refund.

Payment Pending

Trạng thái giao dịch đang chờ kết quả. Rất quan trọng khi gateway chưa trả kết quả cuối cùng.

Payment Unknown

Trạng thái không biết payment thành công hay thất bại, thường do timeout. Không được xem là failed ngay.

Merchant Transaction ID

ID giao dịch phía merchant gửi sang payment gateway. Dùng để query status và chống charge trùng khi retry.

Idempotency Key

Key đảm bảo cùng một request retry nhiều lần vẫn chỉ tạo một kết quả.

Outbox Pattern

Ghi event vào bảng outbox trong cùng transaction với dữ liệu chính, rồi publisher gửi event sau. Tránh mất event.

Inbox Pattern

Service nhận event lưu lại event đã xử lý để chống xử lý trùng. Còn gọi là processed event hoặc idempotent consumer.

Local Transaction

Transaction ngắn trong database của một service. Saga dùng nhiều local transaction thay vì một distributed transaction dài.

Reconciliation

Job đối soát để tìm payment bị kẹt, event chưa publish, wallet chưa credit, refund chưa xong. Rất quan trọng trong payment.



rất tốt, bây giờ hãy giải thích chi tiết về các kiến thức gốc rễ đến nâng nạo trong java/spring boot. có chú thích nếu khó hiểu

Mình sẽ nối tiếp đúng mạch microservice ở trên, nhưng lần này gom lại thành "lộ trình kiến thức Java/Spring Boot từ gốc đến nâng cao", tập trung vào thứ đi phỏng vấn senior và đi làm hệ thống lớn thật sự.

Đã suy nghĩ trong 4s >

Dưới đây là bức tranh **từ gốc rễ đến nâng cao của Java/Spring Boot** theo hướng thực chiến doanh nghiệp, đặc biệt phù hợp với hệ thống banking, payment, e-commerce, microservice traffic cao.

Mình sẽ không giải thích kiểu "học thuộc định nghĩa", mà sẽ đi theo hướng:

Java chạy như thế nào?
Spring Boot quản lý object như thế nào?
Request đi vào hệ thống ra sao?
Transaction hoạt động ra sao?
Làm sao chống lỗi dữ liệu?
Làm sao scale, debug, monitor?
Senior cần tư duy gì?



1. Gốc rễ đầu tiên: Java không chạy trực tiếp trên máy như C/C++

Khi viết Java:

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Hello");  
    }  
}
```

File `.java` sẽ được compile thành `.class`.

```
Main.java --javac--> Main.class
```



File `.class` chứa **bytecode**, không phải machine code trực tiếp.

Sau đó JVM chạy bytecode này.

Java source code

|

v

Bytecode

|

v

JVM

|

v

Machine code



Điểm quan trọng là Java chạy trên **JVM - Java Virtual Machine**.

JVM giống như một “máy ảo” chịu trách nhiệm:

Load class

Quản lý memory

Chạy bytecode

JIT compile code nóng thành machine code

Garbage Collection

Quản lý thread



Vì vậy Java có câu nổi tiếng:

Write once, run anywhere.

Nghĩa là cùng một file bytecode có thể chạy trên Windows, Linux, Mac, miễn là có JVM phù hợp.

2. JVM Memory Model: hiểu memory thì mới debug production tốt

Trong Java, memory quan trọng nhất là:

Heap

Stack

Metaspace

Code Cache



Heap

Heap là vùng nhớ chứa object.

Ví dụ:

```
User user = new User();
```

Object `new User()` nằm trên heap.

Biến `user` là reference trỏ đến object đó.

Stack:
user -----> Heap: User object



Khi hệ thống Spring Boot chạy, hầu hết object như service, repository, DTO, entity, list, map đều nằm trong heap.

Nếu heap đầy, JVM phải chạy Garbage Collection. Nếu GC quá nhiều, hệ thống chậm. Nếu không đủ memory, sẽ bị:

OutOfMemoryError: Java heap space



Stack

Mỗi thread có stack riêng.

Stack chứa:

Method đang chạy
Local variable
Reference
Call chain



Ví dụ:

```
public void process() {  
    int amount = 100;  
    User user = new User();  
}
```

`amount` và biến reference `user` nằm ở stack, còn object `new User()` nằm ở heap.

Nếu method gọi lồng nhau quá sâu, có thể bị:

StackOverflowError



Ví dụ recursion không có điểm dừng:

```
public void call() {  
    call();  
}
```

Metaspace

Metaspace chứa metadata của class.

Ví dụ thông tin class `User`, `PaymentService`, annotation, method, field.

Trong Spring Boot, vì framework load rất nhiều class, proxy, reflection, annotation, nên Metaspace cũng quan trọng.

Code Cache

JVM có JIT compiler. Những đoạn code chạy nhiều sẽ được compile thành machine code tối ưu và lưu ở Code Cache.

3. Garbage Collection: tại sao Java tự dọn rác nhưng vẫn có memory leak?

Java có Garbage Collector nên developer không cần tự free memory như C/C++.

Nhưng không có nghĩa là Java không bị memory leak.

Một object chỉ được GC dọn nếu **không còn reference** nào trỏ đến nó.

Ví dụ memory leak:

```
public class Cache {  
    private static final List<Object> list = new ArrayList<>();  
  
    public void add(Object obj) {  
        list.add(obj);  
    }  
}
```

Nếu list này cứ tăng mãi, object trong list vẫn còn reference, GC không dọn được.

Trong dự án thật, memory leak hay đến từ:

Static collection
 Cache không có eviction
 ThreadLocal không clear
 Listener không unregister
 Map lưu session/user quá lâu
 Load file lớn vào memory
 Query trả quá nhiều records



Ví dụ nguy hiểm:

```
List<Order> orders = orderRepository.findAll();
```

Nếu bảng có 10 triệu record, app có thể chết vì heap đầy.

Best practice là dùng pagination:

```
Page<Order> orders = orderRepository.findAll(PageRequest.of(0, 100));
```

4. Java Object: hiểu reference rất quan trọng

Trong Java, biến object không chứa object thật, nó chứa reference.

Ví dụ:

```
User a = new User("Eric");
User b = a;
b.setName("John");
```

Lúc này a.getName() cũng là "John" .

Vì a và b trỏ cùng một object.

```
a ----\
      ----> User(name="John")
b ----/
```



Đây là lý do khi truyền object vào method, nếu method sửa object đó, bên ngoài cũng thấy thay đổi.

```
public void change(User user) {  
    user.setName("John");  
}
```

Nhưng nếu gán lại reference:

```
public void change(User user) {  
    user = new User("John");  
}
```

thì object bên ngoài không đổi, vì Java pass-by-value đối với reference.

Nói chính xác:

Java luôn pass-by-value. Nhưng value của object variable là reference.

5. String, StringBuilder, StringBuffer

String là immutable.

Nghĩa là sau khi tạo, nội dung không đổi.

```
String s = "abc";  
s = s + "def";
```

Dòng trên không sửa object "abc" , mà tạo object mới "abcdef" .

Nếu nối chuỗi nhiều lần trong loop:

```
String log = "";  
for (int i = 0; i < 10000; i++) {  
    log += i;  
}
```

sẽ tạo rất nhiều object trung gian, tốn memory và GC.

Dùng StringBuilder tốt hơn:

```
StringBuilder builder = new StringBuilder();  
for (int i = 0; i < 10000; i++) {  
    builder.append(i);  
}  
String log = builder.toString();
```

StringBuffer giống StringBuilder nhưng thread-safe vì có synchronized, nên chậm hơn.

Trong thực tế:

Dùng String cho text ít thay đổi.
Dùng StringBuilder khi build string trong một thread.
Dùng StringBuffer rất ít khi cần, vì hiện nay thường quản lý thread-safety ở tầng



Trong logging, không nên nối chuỗi trực tiếp nếu log level có thể bị tắt.

Không tốt:

```
log.info("User " + userId + " paid " + amount);
```

Tốt hơn:

```
log.info("User {} paid {}", userId, amount);
```

Vì logger chỉ format khi cần.

6. BigDecimal: tiền không dùng float/double

Trong banking/payment, không dùng float hoặc double cho tiền.

Vì float/double dùng binary floating point, không biểu diễn chính xác nhiều số thập phân.

Ví dụ:

```
System.out.println(0.1 + 0.2);
```

có thể ra:

```
0.30000000000000004
```



Với tiền, sai 0.000001 cũng nguy hiểm.

Dùng `BigDecimal` :

```
BigDecimal amount = new BigDecimal("100.50");
```

Không nên:

```
BigDecimal amount = new BigDecimal(100.50);
```

Vì `100.50` đã là `double` trước khi đưa vào `BigDecimal`.

Tốt:

```
BigDecimal amount = BigDecimal.valueOf(100.50);
```

hoặc tốt nhất:

```
BigDecimal amount = new BigDecimal("100.50");
```

Khi chia tiền, phải có rounding rõ ràng:

```
amount.divide(new BigDecimal("3"), 2, RoundingMode.HALF_UP);
```

7. Collection: List, Set, Map dùng sai là lỗi performance ngay

List

List cho phép phần tử trùng, có thứ tự.

ArrayList đọc theo index nhanh, thêm cuối nhanh, nhưng insert/delete giữa list chậm.

```
List<String> names = new ArrayList<>();
```

Set

Set không cho trùng.

Dùng khi cần kiểm tra tồn tại nhanh.

```
Set<String> ids = new HashSet<>();
```

Ví dụ chống duplicate transaction ID trong memory batch:

```
if (!processedIds.add(transactionId)) {  
    // duplicate  
}
```

Map

Map lưu key-value.

```
Map<String, User> usersById = new HashMap<>();
```

Tra cứu theo key rất nhanh.

Nhưng muốn dùng object làm key trong HashMap / HashSet, phải override đúng `equals()` và `hashCode()`.

Nếu sai, có thể bị lỗi rất khó debug.

Ví dụ:

```
Set<User> users = new HashSet<>();  
users.add(new User(1L, "A"));  
users.add(new User(1L, "A"));
```

Nếu `equals/hashCode` không đúng, set vẫn chứa 2 user trùng về mặt business.

8. Exception: checked vs unchecked

Java có 2 nhóm exception chính.

Checked Exception

Bắt buộc phải catch hoặc throws.

Ví dụ:

```
IOException  
SQLException
```

Unchecked Exception

Không bắt buộc catch.

Ví dụ:

```
RuntimeException  
NullPointerException  
IllegalArgumentException
```

Trong Spring Boot thực tế, business exception thường là unchecked exception.

Ví dụ:

```
throw new InsufficientBalanceException("Not enough balance");
```

Sau đó dùng `@ControllerAdvice` để map exception thành HTTP response.

```
@RestControllerAdvice  
public class GlobalExceptionHandler {  
  
    @ExceptionHandler({InsufficientBalanceException.class})  
    public ResponseEntity<?> handle(InsufficientBalanceException ex) {  
        return ResponseEntity.status(HttpStatus.BAD_REQUEST)  
    }  
}
```

```

        .body(Map.of("message", ex.getMessage()));
    }
}

```

Best practice:

Không catch exception rồi nuốt mất.
 Không trả stacktrace cho client.
 Log đủ context.
 Business exception nên rõ nghĩa.
 Technical exception nên được wrap hoặc map hợp lý.



Không tốt:

```

try {
    paymentService.pay();
} catch (Exception e) {
    // ignore
}

```

Đây là kiểu lỗi rất nguy hiểm trong production.

9. Thread: nền tảng của request processing

Trong Spring Boot truyền thống dùng Tomcat, mỗi request thường được xử lý bởi một thread từ thread pool.

```

Client request
|
Tomcat thread pool
|
Controller
|
Service
|
Repository
|
Database

```



Nếu hệ thống có 200 thread, tại một thời điểm thường chỉ xử lý được khoảng 200 request đang chạy đồng thời trong app instance đó.

Nếu request bị kẹt vì gọi DB chậm hoặc external API chậm, thread bị giữ lại.

Ví dụ:

```
public PaymentResponse pay() {  
    gatewayClient.call(); // chờ 10 giây  
    return response;  
}
```

Trong 10 giây đó, thread không làm việc khác.

Traffic cao mà nhiều request chờ như vậy, thread pool đầy, request mới phải đợi hoặc timeout.

Đây là lý do cần:

- Timeout
- Connection pool tuning
- Async processing
- Queue
- Circuit breaker
- Bulkhead
- Backpressure



10. Thread-safety: Singleton Bean trong Spring rất dễ bị bug nếu lưu state

Mặc định Spring Bean thường là singleton.

Ví dụ:

```
@Service  
public class PaymentService {  
}
```

Spring chỉ tạo một object `PaymentService` dùng chung cho nhiều request/thread.

Nếu bạn lưu state request vào field, sẽ lỗi.

Sai:

```
@Service
public class PaymentService {

    private String currentUserId;

    public void pay(String userId) {
        this.currentUserId = userId;
        process();
    }
}
```

Vì nhiều request chạy song song có thể ghi đè `currentUserId` .

Đúng hơn:

```
@Service
public class PaymentService {

    public void pay(String userId) {
        process(userId);
    }
}
```

Rule quan trọng:

Spring singleton service nên stateless. Dữ liệu theo request để trong local variable, method parameter, request object, hoặc database.

11. Spring Boot là gì?

Spring Framework cung cấp nền tảng:

- IoC Container
- Dependency Injection
- AOP
- Transaction Management
- MVC
- Data Access
- Security
- Integration



Spring Boot là lớp giúp cấu hình nhanh hơn:

- Auto Configuration
- Embedded Tomcat



Starter dependencies
Actuator
Externalized configuration
Production-ready defaults

Trước đây dùng Spring truyền thống phải config XML rất nhiều.

Spring Boot giúp:

```
@SpringBootApplication
public class App {
    public static void main(String[] args) {
        SpringApplication.run(App.class, args);
    }
}
```

Chạy là có app web với Tomcat embedded.

12. IoC và Dependency Injection: gốc rễ của Spring

Không dùng Spring:

```
public class OrderService {
    private PaymentService paymentService = new PaymentService();
}
```

OrderService tự tạo dependency.

Vấn đề:

Khó test
Khó thay implementation
Coupling cao
Khó quản lý lifecycle



Dùng Spring:

```
@Service
public class OrderService {

    private final PaymentService paymentService;
```

```
public OrderService(PaymentService paymentService) {  
    this.paymentService = paymentService;  
}  
}
```

Spring sẽ tạo `PaymentService`, tạo `OrderService`, rồi inject vào.

Đây là **Dependency Injection**.

Ý tưởng là:

Class không tự tạo dependency. Nó khai báo thứ nó cần, container sẽ cung cấp.

Best practice là dùng constructor injection:

```
@Service  
public class PaymentService {  
  
    private final WalletService walletService;  
  
    public PaymentService(WalletService walletService) {  
        this.walletService = walletService;  
    }  
}
```

Không nên lạm dụng field injection:

```
@Autowired  
private WalletService walletService;
```

Vì khó test, khó thấy dependency bắt buộc, khó dùng final.

13. Bean Lifecycle: Spring tạo object như thế nào?

Khi app start, Spring scan các class có annotation:

```
@Component  
@Service  
@Repository  
@Controller
```



```
@RestController
@Configuration
```

Sau đó Spring tạo bean, inject dependency, gọi lifecycle hook.

Luồng đơn giản:

```
Scan class
|
Create bean object
|
Inject dependencies
|
Post processor
|
Bean ready
```



Ví dụ:

```
@PostConstruct
public void init() {
    // chạy sau khi bean được tạo và inject xong
}
```

Trong thực tế, không nên đặt logic nặng trong @PostConstruct nếu làm app start chậm hoặc dễ fail.

14. Spring Bean Scope

Mặc định là singleton.

```
singleton: một bean dùng chung toàn app
prototype: mỗi lần request bean thì tạo mới
request: mỗi HTTP request một bean
session: mỗi session một bean
```



Trong backend service thông thường:

```
Service: singleton
Repository: singleton
Controller: singleton
DTO/entity: object bình thường, không phải Spring bean
```



Sai lầm hay gặp là biến DTO thành bean hoặc lưu request data trong service singleton.

15. Spring MVC: request đi vào controller như thế nào?

Ví dụ:

```
@RestController
@RequestMapping("/payments")
public class PaymentController {

    private final PaymentService paymentService;

    @PostMapping
    public PaymentResponse create(@RequestBody PaymentRequest request) {
        return paymentService.create(request);
    }
}
```

Request đi qua:

```
Client
|
Tomcat
|
DispatcherServlet
|
Filter
|
Interceptor
|
Controller
|
Service
|
Repository
|
Database
```



DispatcherServlet là trung tâm của Spring MVC. Nó nhận request, tìm controller phù hợp, bind request body vào object, gọi method, rồi convert response thành JSON.

Ví dụ JSON:

```
{
  "amount": 1000000,
```

```
"currency": "VND"  
}
```

Spring dùng Jackson để convert JSON thành object `PaymentRequest` .

16. Validation: đừng để dữ liệu bẩn vào service

Request DTO:

```
public class PaymentRequest {  
  
    @NotNull  
    private BigDecimal amount;  
  
    @NotBlank  
    private String walletId;  
}
```

Controller:

```
@PostMapping  
public PaymentResponse create(@Valid @RequestBody PaymentRequest request) {  
    return paymentService.create(request);  
}
```

Validation nên đặt ở boundary của hệ thống, tức là controller/API layer.

Nhưng business validation vẫn nằm trong service.

Ví dụ:

```
amount không được null: request validation  
số dư có đủ không: business validation  
payment status có cho refund không: business validation
```



17. Layer Architecture trong Spring Boot

Một cấu trúc phổ biến:

Controller
|
Service
|
Repository
|
Database



Controller

Nhận request, validate input cơ bản, trả response.

Không nên chứa business logic phức tạp.

Service

Chứa business logic.

Ví dụ:

Tạo payment
Kiểm tra idempotency
Trừ ví
Ghi ledger
Publish event



Repository

Làm việc với database.

DTO

Dữ liệu request/response.

Entity

Mapping với database table.

Không nên trả entity trực tiếp ra API. Nên map entity sang response DTO.

Vì entity có thể chứa field nhạy cảm, quan hệ lazy loading, hoặc thay đổi schema nội bộ.

18. JPA/Hibernate: tiện nhưng phải hiểu bản chất

JPA là specification. Hibernate là implementation phổ biến.

Entity:

```
@Entity
@Table(name = "payments")
public class Payment {

    @Id
    private String id;

    private BigDecimal amount;

    @Enumerated(EnumType.STRING)
    private PaymentStatus status;
}
```

Repository:

```
public interface PaymentRepository extends JpaRepository<Payment, String> {
}
```

Khi gọi:

```
paymentRepository.save(payment);
```

Hibernate sẽ generate SQL.

Nhưng senior phải hiểu Hibernate không phải magic. Nó có:

- Persistence Context
- Dirty Checking
- Lazy Loading
- Flush
- Transaction
- N+1 Query problem
- First-level cache



19. Persistence Context và Dirty Checking

Trong một transaction:

```
@Transactional
public void updatePayment(String id) {
    Payment payment = paymentRepository.findById(id).orElseThrow();
    payment.setStatus(PaymentStatus.SUCCESS);
}
```

Không gọi `save()` nhưng Hibernate vẫn update DB.

Vì entity đang được quản lý trong Persistence Context. Hibernate thấy object thay đổi, cuối transaction tự flush update.

Đây gọi là **Dirty Checking**.

Nhưng nếu object detached, dirty checking không hoạt động.

20. Lazy Loading và lỗi hay gặp

Quan hệ:

```
@OneToMany(mappedBy = "order", fetch = FetchType.LAZY)
private List<OrderItem> items;
```

Lazy nghĩa là chưa load `items` ngay khi load `Order`.

Khi gọi:

```
order.getItems()
```

Hibernate mới query DB.

Nếu transaction/session đã đóng, có thể lỗi:

LazyInitializationException



Best practice:

Không trả entity trực tiếp ra controller
Dùng fetch join/entity graph khi cần
Map sang DTO trong transaction
Cẩn thận với Open Session In View



21. N+1 Query Problem

Ví dụ:

```
List<Order> orders = orderRepository.findAll();

for (Order order : orders) {
    System.out.println(order.getItems().size());
}
```

Nếu có 100 orders:

1 query lấy orders
100 query lấy items cho từng order



Tổng 101 query. Đây là N+1.

Fix bằng fetch join:

```
@Query("select o from Order o join fetch o.items where o.id in :ids")
List<Order> findWithItems(@Param("ids") List<Long> ids);
```

Hoặc dùng @EntityGraph .

Trong production, N+1 có thể làm API từ 100ms thành 10 giây.

22. Transaction trong Spring: @Transactional hoạt động thật sự thế nào?

Khi viết:

```
@Transactional
public void createPayment() {
    // business logic
}
```

Spring không nhét transaction trực tiếp vào method. Nó tạo proxy bọc quanh bean.

Luồng:

```

Client gọi proxy
|
Proxy mở transaction
|
Gọi method thật
|
Nếu success: commit
Nếu exception: rollback

```



Đây là AOP.

Điểm rất quan trọng: `@Transactional` chỉ có tác dụng khi method được gọi qua Spring proxy.

Sai lầm:

```

@Service
public class PaymentService {

    public void outer() {
        inner();
    }

    @Transactional
    public void inner() {
        // transaction may not work as expected
    }
}

```

Đây là self-invocation. Method `inner()` được gọi nội bộ trong cùng class, không đi qua proxy, nên annotation có thể không hoạt động như mong đợi.

Cách xử lý:

```

Đưa method transactional sang bean khác
Hoặc đặt @Transactional ở outer
Hoặc dùng TransactionTemplate khi cần kiểm soát rõ

```



23. Rollback rule trong Spring Transaction

Mặc định Spring rollback với unchecked exception:

```

RuntimeException

```



Error

Nhưng checked exception thì không rollback mặc định.

Ví dụ:

```
@Transactional
public void process() throws IOException {
    saveSomething();
    throw new IOException();
}
```

Có thể transaction không rollback nếu không cấu hình.

Muốn rollback:

```
@Transactional(rollbackFor = Exception.class)
public void process() throws Exception {
}
```

Trong business service, thường dùng custom runtime exception để rõ ràng.

24. Transaction Propagation

Propagation quyết định transaction mới dùng transaction hiện tại hay tạo transaction mới.

Phổ biến nhất:

REQUIRED

Mặc định.

Nếu đã có transaction thì dùng tiếp. Nếu chưa có thì tạo mới.

```
@Transactional(propagation = Propagation.REQUIRED)
```

REQUIRES_NEW

Luôn tạo transaction mới, suspend transaction hiện tại.

Dùng khi muốn một phần commit độc lập.

Ví dụ audit log:

```
@Transactional(propagation = Propagation.REQUIRES_NEW)
public void saveAuditLog() {
}
```

Nhưng phải cẩn thận. Nếu transaction chính rollback, audit log vẫn commit.

MANDATORY

Bắt buộc phải có transaction sẵn, nếu không thì lỗi.

NEVER

Không được chạy trong transaction.

25. Isolation Level

Isolation level quyết định transaction nhìn thấy dữ liệu của transaction khác như thế nào.

Hay gặp:

```
READ_COMMITTED
REPEATABLE_READ
SERIALIZABLE
```



Trong nhiều hệ thống, dùng READ_COMMITTED rồi kết hợp:

```
Unique constraint
Atomic update
Optimistic locking
Pessimistic locking
```



Ví dụ trừ tiền an toàn:

```
UPDATE wallets
SET balance = balance - :amount
WHERE wallet_id = :walletId
AND balance >= :amount;
```

Sau đó check row affected .

Đây tốt hơn nhiều so với đọc balance ra Java rồi tự tính lòng lẻo.

26. Optimistic Locking trong JPA

Entity:

```
@Entity
public class Wallet {

    @Id
    private String id;

    private BigDecimal balance;

    @Version
    private Long version;
}
```

Hibernate sẽ update kèm version.

Nếu hai transaction cùng update một wallet, một cái thành công, cái sau fail vì version đã đổi.

Phù hợp khi conflict không quá cao.

Nếu conflict cao, retry quá nhiều có thể làm hệ thống chậm. Khi đó cân nhắc atomic SQL, queue theo key, hoặc pessimistic locking.

27. Pessimistic Locking trong JPA

Repository:

```
@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("select w from Wallet w where w.id = :id")
Optional<Wallet> findByIdForUpdate(@Param("id") String id);
```

Nó tương đương lock row ở database.

Dùng khi cần đảm bảo không ai sửa dòng đó trong transaction.

Nhưng không được giữ lock lâu, không gọi external API trong transaction đang lock.

28. REST API Design trong Spring Boot

Một API tốt không chỉ là chạy được. Nó phải rõ ràng, predictable, dễ maintain.

Ví dụ resource payment:

```
POST /payments
GET /payments/{paymentId}
POST /payments/{paymentId}/refunds
```

Status code:

```
200 OK: request thành công
201 Created: tạo resource thành công
202 Accepted: nhận request, xử lý async
400 Bad Request: input sai
401 Unauthorized: chưa đăng nhập
403 Forbidden: không có quyền
404 Not Found: không tìm thấy
409 Conflict: conflict/idempotency duplicate khác payload
500 Internal Server Error: lỗi server
```



Với Saga async, thường trả:

```
202 Accepted
```

Response:

```
{
  "paymentId": "pay-001",
  "status": "PROCESSING"
}
```

29. Idempotency trong Spring Boot

Controller:

```
@PostMapping("/payments")
public ResponseEntity<PaymentResponse> create(
    @RequestHeader("Idempotency-Key") String idempotencyKey,
    @Valid @RequestBody PaymentRequest request
) {
    PaymentResponse response = paymentService.create(idempotencyKey, request);
    return ResponseEntity.accepted().body(response);
}
```

Service logic:

1. Tính request hash
2. Check idempotency key đã tồn tại chưa
3. Nếu tồn tại và hash giống: trả response cũ
4. Nếu tồn tại và hash khác: trả 409 Conflict
5. Nếu chưa tồn tại: tạo record PROCESSING
6. Xử lý business
7. Lưu response SUCCESS/FAILED



Database cần unique:

```
CREATE UNIQUE INDEX uk_idempotency_key
ON idempotency_records(idempotency_key);
```

Code phải bắt duplicate key vì hai request có thể đến cùng lúc.

30. Spring Security: Authentication vs Authorization

Authentication

Bạn là ai?

Ví dụ user login bằng username/password hoặc JWT.

Authorization

Bạn được phép làm gì?

Ví dụ user thường không được gọi API admin.

Spring Security dùng filter chain.

Request đi vào:

```

Client
|
Security Filters
|
Authentication
|
Authorization
|
Controller

```



JWT flow phổ biến:

```

Client gửi Authorization: Bearer <token>
Filter đọc token
Validate signature/expiration
Lấy user info/roles
Set Authentication vào SecurityContext
Controller xử lý

```



Ví dụ method security:

```

@PreAuthorize("hasRole('ADMIN')")
public void deleteUser(String userId) {
}

```

Best practice:

```

Không lưu secret trong code
Token phải có expiration
Refresh token phải quản lý cẩn thận
Password phải hash bằng BCrypt/Argon2
Không log token/password
API nội bộ cũng cần auth nếu môi trường zero-trust

```



31. Configuration trong Spring Boot

Không hardcode config.

Sai:

```
String url = "jdbc:postgresql://prod-db:5432/app";
```

Đúng:

```
spring:
  datasource:
    url: ${DB_URL}
    username: ${DB_USERNAME}
    password: ${DB_PASSWORD}
```

Dùng profile:

```
application-dev.yml
application-staging.yml
application-prod.yml
```



Chạy:

```
-Dspring.profiles.active=prod
```

hoặc environment variable.

Secrets nên để trong secret manager, Kubernetes secret, Vault, AWS Secrets Manager, etc.

32. Connection Pool: bottleneck hay bị quên

Spring Boot thường dùng HikariCP.

Database connection là tài nguyên đắt.

Nếu mỗi request tạo connection mới sẽ rất chậm. Connection pool giữ sẵn một số connection.

Ví dụ:

```
spring:
  datasource:
    hikari:
      maximum-pool-size: 20
```

Nếu app có 20 DB connections nhưng Tomcat có 200 threads, thì có thể 200 requests vào cùng lúc nhưng chỉ 20 request dùng DB được tại một thời điểm, còn lại chờ connection.

Nếu DB query chậm, connection bị giữ lâu, pool cạn, API timeout.

Senior phải biết nhìn:

```
DB connection pool active
DB connection pool idle
Connection wait time
Slow query
Thread pool usage
```



Không phải cứ tăng connection pool là tốt. DB cũng có giới hạn.

33. Timeout: không có timeout là tự sát production

Mọi external call phải có timeout.

Ví dụ gọi service khác bằng WebClient/RestTemplate/OpenFeign mà không set timeout, request có thể treo rất lâu.

Cần có:

```
Connection timeout
Read timeout
Request timeout
Database query timeout
Transaction timeout
Kafka consumer timeout
```



Ví dụ nguyên tắc:

```
User-facing API timeout: 2-5s tùy nghiệp vụ
Internal service call: ngắn hơn timeout tổng
Gateway/payment: có timeout nhưng xử lý UNKNOWN/PENDING
```



Không nên để một request giữ thread 60 giây nếu client đã timeout sau 5 giây.

34. Retry: dùng sai là nhân đôi traffic

Retry cần thiết nhưng nguy hiểm.

Không retry mọi lỗi.

Nên retry:

Timeout tạm thời
Connection reset
503 Service Unavailable
429 Too Many Requests nếu có backoff



Không nên retry:

400 Bad Request
401 Unauthorized
403 Forbidden
Business error như insufficient balance
Validation error



Retry phải có:

Max attempts
Exponential backoff
Jitter
Idempotency key



Ví dụ retry payment mà không có idempotency có thể charge trùng.

35. Circuit Breaker

Khi một service downstream đang chết, nếu upstream cứ gọi liên tục thì toàn hệ thống chậm theo.

Circuit breaker có 3 trạng thái:

CLOSED: gọi bình thường
OPEN: chặn call, fail fast
HALF_OPEN: thử một ít request xem service hồi chưa



Ví dụ Payment Service gọi Fraud Service. Fraud Service down.

Nếu không có circuit breaker:

Request chờ timeout hàng loạt
Thread bị giữ
Hệ thống nghẽn



Có circuit breaker:

Fail fast
Trả response degraded hoặc chuyển async
Giữ hệ thống sống



Trong Spring Boot thường dùng Resilience4j.

36. Bulkhead

Bulkhead nghĩa là chia tài nguyên để lỗi một phần không kéo chết toàn hệ thống.

Ví dụ:

Payment API dùng thread pool riêng
Notification API dùng thread pool riêng
Report API dùng thread pool riêng



Nếu report rất chậm, nó không ăn hết thread của payment.

Trong hệ thống lớn, bulkhead giúp cô lập sự cố.

37. Cache: nhanh hơn nhưng dễ sai dữ liệu

Cache dùng để giảm tải DB.

Ví dụ:

```
@Cacheable(value = "users", key = "#userId")  
public UserResponse getUser(String userId) {  
    return userRepository.findById(userId);  
}
```

Nhưng cache có các vấn đề:

Cache stale: dữ liệu cũ
Cache invalidation khó
Cache stampede
Cache penetration
Cache avalanche



Không nên cache dữ liệu tài chính nhạy cảm như balance khả dụng nếu không có chiến lược cực kỳ rõ.

Có thể cache:

```
Config
Product catalog
User profile ít đổi
Permission metadata
Reference data
```



Cẩn thận khi cache:

```
Balance
Inventory
Payment status
Security permission
```



38. Async Processing trong Spring

Có thể dùng:

```
@Async
public void sendEmail() {
}
```

Nhưng @Async không phải magic. Nó dùng thread pool.

Phải config executor:

```
@Bean
public Executor taskExecutor() {
    ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
    executor.setCorePoolSize(10);
    executor.setMaxPoolSize(50);
    executor.setQueueCapacity(1000);
    executor.initialize();
    return executor;
}
```

Nếu không kiểm soát queue, có thể memory tăng cao.

Trong nghiệp vụ quan trọng, thay vì @Async trong memory, nên dùng message queue như Kafka/RabbitMQ/SQS để có durability.

Vì nếu app crash, task @Async trong memory mất.

39. Message Queue với Spring Boot

Trong microservice, queue giúp decouple service.

Ví dụ:

```
Payment Service publish PaymentSucceeded
Wallet Service consume
Ledger Service consume
Notification Service consume
```



Lợi ích:

```
Service không cần gọi nhau trực tiếp quá nhiều
Retry dễ hơn
Xử lý async
Chịu tải tốt hơn
```



Nhưng phải nhớ:

```
Message có thể trùng
Message có thể delay
Consumer có thể fail
Ordering không phải lúc nào cũng global
```



Consumer phải idempotent.

40. Kafka trong hệ thống lớn

Kafka dùng topic, partition, consumer group.

```
Topic: payment-events
Partition: 0, 1, 2, 3
Consumer Group: wallet-service
```



Message cùng key sẽ vào cùng partition, giữ được ordering trong partition đó.

Ví dụ key là walletId , thì event của cùng wallet đi cùng partition.

```
key = wallet:A
```



Consumer group giúp scale consumer. Nhưng trong cùng một group, một partition chỉ được consume bởi một consumer tại một thời điểm.

Điều này giúp parallel processing nhưng vẫn giữ ordering theo partition.

41. Outbox Pattern trong Spring Boot

Khi service update DB và publish event, không publish trực tiếp sau save nếu event quan trọng.

Sai:

```
paymentRepository.save(payment);  
kafkaTemplate.send("payment-events", event);
```

Nếu DB save thành công nhưng Kafka send fail, event mất.

Đúng:

```
BEGIN TRANSACTION
```

1. Save payment
2. Save outbox event

```
COMMIT
```



Sau đó worker đọc outbox và publish.

Entity outbox:

```
@Entity  
public class OutboxEvent {  
    @Id  
    private String eventId;  
  
    private String aggregateId;  
  
    private String eventType;  
  
    @Lob
```

```
private String payload;  
  
private String status;  
}
```

Worker:

```
Đọc event NEW  
Publish Kafka  
Mark PUBLISHED
```



Trong production có thể dùng Debezium CDC để đọc outbox.

42. Saga trong Spring Boot

Saga thường được implement bằng:

```
Orchestrator service  
State machine  
Message queue  
Outbox/inbox  
Retry  
Compensation
```



Ví dụ payment top-up:

```
Payment CREATED  
Payment PENDING  
Gateway SUCCESS  
Wallet CREDIT_PENDING  
Wallet CREDITED  
Ledger WRITTEN  
Payment COMPLETED
```



Không nên giữ transaction dài qua nhiều service.

Mỗi service chỉ commit local transaction của nó.

Nếu bước sau fail, dùng compensation.

Ví dụ:

```
Gateway charge success  
Wallet credit fail permanent  
=> refund payment
```



43. Testing trong Spring Boot

Senior không chỉ biết viết code, mà phải biết test đúng tầng.

Unit Test

Test một class, mock dependency.

```
@ExtendWith(MockitoExtension.class)
class PaymentServiceTest {
}
```

Dùng cho business logic.

Integration Test

Test với Spring context, database thật hoặc Testcontainers.

```
@SpringBootTest
class PaymentIntegrationTest {
}
```

Repository Test

```
@DataJpaTest
class PaymentRepositoryTest {
}
```

Controller Test

```
@WebMvcTest(PaymentController.class)
class PaymentControllerTest {
}
```

Contract Test

Dùng khi microservice giao tiếp với nhau, đảm bảo API contract không vỡ.

Load Test

Dùng để test traffic cao, thread pool, DB pool, latency.

Trong hệ thống payment, test quan trọng:

Duplicate request
Concurrent request
Retry after timeout
Message duplicate
Outbox publish fail
Consumer crash
Saga compensation
DB deadlock/retry



44. Observability: log, metrics, tracing

Production không debug bằng cách "đoán".

Cần có:

Structured log
Metrics
Distributed tracing
Alert
Dashboard



Log nên có:

requestId
correlationId
traceId
userId
transactionId
paymentId
eventId
idempotencyKey



Ví dụ log tốt:

paymentId=pay-001 walletId=A status=PAYMENT_SUCCEEDED correlationId=corr-123



Không tốt:

success



Metrics cần xem:

Request rate
Error rate
Latency p95/p99
DB connection usage
Thread pool usage
Kafka lag
Outbox pending
DLQ count
GC pause
Memory usage
CPU usage



Tracing giúp thấy request đi qua service nào:

API Gateway -> Payment Service -> Wallet Service -> Ledger Service



Spring Boot Actuator hỗ trợ endpoint production như health, metrics.

45. Performance tuning trong Java/Spring Boot

Khi API chậm, đừng đoán. Phải tìm bottleneck.

Có thể chậm ở:

DB query
N+1 query
External API
Thread pool full
Connection pool full
GC pause
Serialization JSON
Lock contention
Queue lag
CPU bound logic



Best practice:

Pagination
Index database đúng
Không query thừa
Không load object quá lớn
Dùng projection DTO
Batch insert/update khi cần
Timeout external call
Cache dữ liệu đọc nhiều ít đổi
Monitor p95/p99 latency



Ví dụ không nên:

```
List<User> users = userRepository.findAll();
```

cho bảng lớn.

Nên:

```
Page<User> users = userRepository.findAll(PageRequest.of(page, size));
```

46. Database Index: backend senior phải hiểu

Query:

```
SELECT * FROM payments WHERE user_id = ? AND status = ? ORDER BY created_at DESC;
```

Nên có index phù hợp:

```
CREATE INDEX idx_payments_user_status_created  
ON payments(user_id, status, created_at DESC);
```

Không có index, DB phải scan nhiều row, API chậm.

Nhưng index cũng có cost:

Tốn storage
Insert/update chậm hơn
Quá nhiều index làm write chậm



Senior cần biết đọc execution plan, slow query log.

47. API versioning

Khi API đã có client dùng, không thể sửa breaking change tùy tiện.

Ví dụ:

```
/api/v1/payments  
/api/v2/payments
```

Hoặc dùng header version.

Không nên xóa field cũ ngay. Nên deprecate trước.

Trong microservice lớn, API contract rất quan trọng.

48. Clean Code trong Java/Spring Boot

Code tốt không phải là code “ngắn nhất”, mà là code dễ hiểu, dễ test, dễ thay đổi.

Nguyên tắc:

```
Controller mỏng  
Service rõ business  
Method không quá dài  
Tên biến/method có nghĩa  
Không duplicate logic  
Không throw Exception chung chung  
Không dùng Map<String, Object> lung tung cho business core  
Không để entity leak ra API
```



Ví dụ tốt:

```
paymentService.createTopup(command);
```

Tốt hơn:

```
service.process(data);
```

Tên method phải nói được nghiệp vụ.

49. Design Pattern thường gặp trong Spring Boot

Factory

Tạo object theo loại.

Ví dụ chọn payment gateway theo provider:

```
PaymentGateway gateway = gatewayFactory.get(provider);
```

Strategy

Nhiều thuật toán có thể thay thế nhau.

Ví dụ fee calculation:

```
DomesticFeeStrategy  
InternationalFeeStrategy  
PremiumUserFeeStrategy
```



Template Method

Định nghĩa skeleton flow, subclass customize một số bước.

Adapter

Bọc API bên ngoài thành interface nội bộ.

Ví dụ:

```
public interface PaymentGatewayClient {  
    ChargeResult charge(ChargeCommand command);  
}
```

Implementation cho Stripe, PayPal, VNPay khác nhau.

Decorator

Thêm behavior như logging, retry, metrics quanh service.

Builder

Tạo object phức tạp dễ đọc hơn.

50. Senior mindset: biết trade-off

Junior thường hỏi:

Dùng Redis lock hay DB lock?
 Dùng Kafka hay REST?
 Dùng JPA hay JDBC?
 Dùng sync hay async?
 Dùng monolith hay microservice?



Senior sẽ hỏi lại:

Traffic bao nhiêu?
 Data correctness cần mức nào?
 Latency yêu cầu bao nhiêu?
 Có retry không?
 Có duplicate không?
 Có cần ordering không?
 Fail thì recover thế nào?
 Ai là source of truth?
 Có audit không?
 Team vận hành được không?



Không có pattern nào luôn đúng.

Ví dụ:

REST đơn giản hơn Kafka nếu flow sync và nhỏ.
 Kafka tốt hơn nếu cần async, fan-out, retry, decoupling.
 JPA nhanh dev nhưng dễ N+1 nếu không hiểu.
 JDBC/MyBatis kiểm soát SQL tốt hơn trong query phức tạp.
 Redis lock tốt cho cache/job, nhưng không thay thế DB transaction.
 Saga tốt cho distributed transaction, nhưng làm hệ thống phức tạp hơn.



51. Một kiến trúc Spring Boot thực tế cho payment service

Có thể tổ chức như sau:

```
payment-service
├── api
│   ├── PaymentController
│   ├── request
│   └── response
├── application
│   ├── PaymentApplicationService
│   ├── command
│   └── usecase
└── domain
```



```

|   |   | Payment
|   |   | PaymentStatus
|   |   | PaymentDomainService
|   |   | exception
|   |
|   |--- infrastructure
|   |   | persistence
|   |   | messaging
|   |   | gateway
|   |   | config
|   |
|   |--- common
|       | error
|       | logging
|       | security

```

Ý tưởng:

api: nhận HTTP
 application: điều phối use case
 domain: business core
 infrastructure: database, Kafka, external API
 common: phần dùng chung



Không nên để toàn bộ code dồn vào một service class 3000 dòng.

52. Checklist senior Java/Spring Boot nên nắm

Nếu đi phỏng vấn senior, bạn nên nói được sâu các nhóm này:

JVM memory, GC, thread
 Java collection, concurrency, exception
 BigDecimal cho tiền
 Spring IoC, DI, Bean lifecycle
 Spring MVC request flow
 Spring Transaction, proxy, propagation, isolation
 JPA/Hibernate, lazy loading, dirty checking, N+1
 REST API design
 Security/JWT
 Connection pool, timeout, retry
 Idempotency
 Outbox/Inbox
 Kafka/message queue
 Saga/compensation
 Distributed lock đúng cách
 Testing strategy
 Observability
 Performance tuning



Database index
Clean architecture
Trade-off system design

53. Tóm tắt dễ nhớ

Java/Spring Boot không chỉ là biết annotation.

Một backend engineer mạnh phải hiểu từ dưới lên:

JVM quản lý memory/thread như thế nào
Java object/reference/concurrency hoạt động ra sao
Spring tạo bean và inject dependency như thế nào
Request đi qua controller/service/repository ra sao
Transaction thật sự commit/rollback thế nào
Database bảo vệ dữ liệu bằng constraint/lock/index ra sao
Message queue có thể duplicate như thế nào
Microservice fail/retry/timeout ra sao
Hệ thống production monitor/debug bằng gì



Nói ngắn gọn:

Junior biết viết API chạy được.

Middle biết tổ chức code, transaction, database, test.

Senior biết hệ thống sẽ fail ở đâu, dữ liệu sai ở đâu, traffic cao nghẽn ở đâu, và thiết kế để hệ thống vẫn recover được.

Danh sách thuật ngữ / keyword quan trọng

1. JVM

Java Virtual Machine. Máy ảo chạy bytecode Java, quản lý memory, GC, thread, class loading.

2. Bytecode

Mã trung gian sau khi compile Java source. JVM đọc bytecode và chạy.

3. Heap

Vùng nhớ chứa object trong Java.

4. Stack

Vùng nhớ riêng của từng thread, chứa local variable và call stack.

5. Garbage Collection

Cơ chế JVM tự dọn object không còn được reference.

6. Memory Leak

Tình trạng object không còn cần dùng nhưng vẫn bị giữ reference nên GC không dọn được.

7. Immutable

Object không thể thay đổi sau khi tạo. `String` là immutable.

8. BigDecimal

Kiểu dữ liệu dùng cho số thập phân chính xác, đặc biệt là tiền.

9. Thread Pool

Tập hợp thread được tái sử dụng để xử lý task/request.

10. Thread-safety

Khả năng code chạy đúng khi nhiều thread truy cập cùng lúc.

11. IoC

Inversion of Control. Thay vì class tự tạo dependency, container quản lý và cung cấp dependency.

12. Dependency Injection

Cơ chế inject object cần dùng vào class, thường qua constructor.

13. Bean

Object được Spring Container quản lý.

14. Singleton Bean

Bean chỉ có một instance trong Spring context, dùng chung cho nhiều request.

15. DispatcherServlet

Thành phần trung tâm của Spring MVC, nhận request và route đến controller.

16. DTO

Data Transfer Object. Object dùng để nhận request hoặc trả response, không nhất thiết mapping DB.

17. Entity

Object mapping với database table trong JPA/Hibernate.

18. Persistence Context

Vùng Hibernate quản lý entity trong một transaction/session.

19. Dirty Checking

Hibernate tự phát hiện entity thay đổi và update DB khi transaction flush/commit.

20. Lazy Loading

Chỉ load dữ liệu liên quan khi thật sự truy cập.

21. N+1 Query

Lỗi performance khi một query chính kéo theo N query phụ.

22. @Transactional

Annotation để Spring quản lý transaction thông qua proxy/AOP.

23. Transaction Propagation

Quy tắc method dùng transaction hiện tại hay tạo transaction mới.

24. Isolation Level

Mức độ cô lập giữa các transaction trong database.

25. Optimistic Locking

Cơ chế dùng version để phát hiện update conflict.

26. Pessimistic Locking

Cơ chế lock dữ liệu để transaction khác không sửa cùng lúc.

27. Idempotency

Cùng một request xử lý nhiều lần nhưng kết quả cuối cùng chỉ như một lần.

28. Outbox Pattern

Ghi event vào DB cùng transaction với business data, sau đó publish ra queue.

29. Inbox Pattern

Consumer lưu event đã xử lý để chống duplicate message.

30. Saga

Pattern chia transaction lớn qua nhiều service thành nhiều local transaction nhỏ, có retry và compensation.

31. Compensation

Hành động bù khi một bước trong saga đã thành công nhưng bước sau fail.

32. Circuit Breaker

Cơ chế ngắt gọi service lỗi để tránh kéo sập toàn hệ thống.

33. Bulkhead

Cô lập tài nguyên để một phần hệ thống lỗi không ảnh hưởng toàn bộ.

34. Connection Pool

Pool connection DB được tái sử dụng để tránh tạo connection mới liên tục.

35. Observability

Khả năng quan sát hệ thống qua log, metrics, tracing, alert.

36. Correlation ID

ID dùng để trace một request xuyên qua nhiều service.

37. Kafka Partition

Phần chia nhỏ của Kafka topic, giúp scale và giữ ordering trong cùng partition.

38. Consumer Group

Nhóm consumer cùng xử lý một topic, giúp scale việc consume message.

39. Cache Stampede

Nhiều request cùng rebuild cache khi cache hết hạn, gây spike DB.

40. Backpressure

Cơ chế giảm/giới hạn tải đầu vào khi hệ thống phía sau quá tải.

